Ecological theory for the biodiversity crisis

Henrique Miguel Pereira



2026-10-12

Ecological theory for the biodiversity crisis

Henrique Miguel Pereira

Table of contents

Preface **1**

I Ecology of individuals 3

1 Ecophysiology and the climate space: when to bask in the sun? 4
1.1 A model for the body temperature of an animal 4
1.2 Understanding the climate space 8
1.3 From ecophysiological models to species distribution models 10
1.4 Statistical confrontation: regressing the climate space 16

2 Economic models of behavior: do animals optimize? 22
2.1 Searching predator 22
2.2 Sit-and-wait predator 28
2.3 Does natural selection maximizes fitness? 34
2.4 Statistical confrontation: finding the maximum 34

3 Conflict resolution in ecology and evolution: war and peace 35
3.1 The prisoner's dillema and the evolution of cooperation 36
3.2 The Hawk-Dove game and animal contests 37
3.3 The war of atttrition 37

4 Movement and dispersal: should I stay or should I go? 38
4.1 The random-walk 38
4.2 Habitat selection 39

II Ecology of populations 40

5 Demography: boom or bust 41

III Ecology of communities 42

6 Measuring biodiversity across scales 43

6.1 Species-area relationship 44
6.2 Species abundance distributions 44
6.3 Estimating diversity indices 44

7 Modelling biodiversity change: winners and losers 45

References 46

Appendices 48

A A brief R tutorial 48
A.1 Basic data structures 48
A.2 Basic plots in R 53
A.3 Iterations and conditional expressions 56
A.4 Writing functions 60
A.5 Random numbers and statistical distributions 62
A.6 Intro to apply, pipes, ggplot2, tidyverse 64
A.7 Working with biodiversity data: GBIF, EBV Portal 80
A.8 Working with raster and shapefiles 82

B Computational labs 83
B.1 Ecophysiology of an ectotherm 83
B.2 Optimal foraging theory 85
B.3 Evolutionary games 86
B.4 Dispersal and the random-walk 88
B.5 Pandemic growth 89
B.6 Population Viability Analysis 90
B.7 Reference 90
B.8 The camera trapper 91
B.9 Managing a fishery 92
B.10 Road kill 93
B.11 The unified neutral theory of biodiversity and biogeography 95
B.12 Monitoring biodiversity 101

Preface

This book is under construction.

We live in the midst of the biodiversity crisis. Biodiversity science, at first mostly a spin off from Ecology, has developed tremendously in the last two decades, becoming a truly interdisciplinary field, with contributions from Geography, Economy, Social Sciences and many other disciplines. Big data and the development of statistical ecology has also been remarkable. As I write these lines, the Global Biodiversity Information Facility or GBIF (https://gbif.org) is about to hit 3 billion records. The availability of large amounts of data and the availability of the open source R software (https://www.r-project.org) has made statistical approaches take the main stage in much of ecological research. However, at the same time, as a biodiversity researcher, practitioner, and a teacher, I find again and again the need for a good understanding of ecological theory to be able to carry out my work. This book if for all of you that have also felt a similar need.

Programming and mathematics are key tools of ecological theory. The programming language I use in this book is R (version 4, available from https://www.r-project.org). If you have familiarly with programming in another computer language, a fast skimming of Appendix A will be enough to get you going with the book. If you are new to programming, I recommend you spend a few hours working through the short course in Appendix A, but you may also wanna consult some web resources (e.g. https://ourcodingclub.github.io/course). With the advent of Large Language Models (LLM) such as ChatGPT there may be the temptation to think programming knowledge will no longer be needed. I think LLMs will accelerate our writing of computer code, but they do not (yet) replace the need for being able to read and understand the code.

You may wonder why I chose **R**. There are plenty other programming languages out there, several suitable for ecological theory and modelling (e.g. data scientists love Pyhton). However, **R** has become the *de facto* language of ecologists with dozens of packages[1] for a wide range of analysis, from camera trapping to species richness estimation. **R** has also powerful packages for spatial data analysis and can work as a Geographic Information System. So R, grew from a statistics software (the origins of R are the comercial S-PLUS) to a general purpose programming language with dedicated packages for different uses.

Why was R so successful, to the point of making some commercial software companies go out of business? R was open-source and free from its beginning, part of the free and open-source movement that gained popularity in the 1990's, particularly with the development of the operating system Linux. Open-source means that the code behind R (R itself is a computer program written in

[1] An R package is a set of functions developed by someone to extend the basic functionality of R.

C, Fortran, and other languages) is available for anyone to study. It also allows for anyone to contribute to improve the code in a collaborative way, including by contributing with packages. R is also free, meaning that anyone can download the software or any of its packages without any cost. Isn't that a wonderful tool for science? Scientists dedicate their life to the pursue of knowledge, and the idea that someone has to pay for accessing that knowledge has always been a bit apocryphal. So R allowed scientists to share their research and code for free in an integrated platform. Another advantage of open-source software, is that we can in theory ran programs that we wrote decades ago by downloading archived versions of the software. At least in theory, the practice of code reproducibility in science is a bit more complex and requires careful data and code management by the researcher[2]. Platforms such as GitHub (https://github.com) for sharing code and Zenodo (https://zenodo.org) or Dryad (https://datadryad.org) for sharing data are a key component of reproducible science.

In contrast a lot of the code I wrote for my PhD was based on a commercial package (Mathematica) and anyone that wants to reuse it a couple decades later has to pay a few hundred Euros for the package (well there are student discounts) and has to find a version of the Mathematica that still runs this code. This latter problem can be insurmountable, as sometimes older versions of the software are no longer commercially available. One can argue that many programming languages are also free and public, however for ecological theory and modelling a high-level language like R provides a nice integrated platform for development and is literally priceless.

But programming is not enough. In order to be able to effectively develop theory and models of biodiversity, a good grasp of mathematics, including of probability and statistics is essential. One challenge is that, and this at least the case in Europe where I have lectured the most, most biology students receive very little mathematical training. This book cannot address by itself such gap and assumes some basic familiarity with probability theory (e.g. what is a probability distribution function), some calculus training (e.g. what area a derivative and an integral), a little algebra (e.g. multiplication of a matrix by a vector). I try to take the reader forward from that level. There are excellent books out there for those that need this basic background (e.g. Otto's and Day's A Biologist's Guide to Mathematical Modelling (2007) for math and Mangel's and Hilborn's Ecological Detective (Hilborn and Mangel 1997) for statistics). And Wikipedia (https://www.wikipedia.org) is often your best friend when you wanna a fast refresh of any mathematical concept or even for many of the models and ecological concepts presented in this book.

This book can be used as the support for a semester course in Ecological Theory or Ecological Modelling. I have used many of these materials over the last decade in a similar course at iDiv/University of Halle-Wittenberg. But the book can also be used as self-learning tool or even as a reference tool. This book takes a lot of inspiration from the Primer of Ecological Theory of Joan Roughgarden (1998a) (the first two chapters draw heavily on her first chapters), that I was lucky to have as a mentor. As she used to say, one writes papers for the reviewers and books for the readers. This book is for you.

[2]For a guide on best practices for reproducible code see https://www.britishecologicalsociety.org/wp-content/uploads/2019/06/BES-Guide-Reproducible-Code-2019.pdf

Part I

Ecology of individuals

Chapter 1

Ecophysiology and the climate space: when to bask in the sun?

We are now almost daily bombarded with news about climate change and its impacts on people and ecosystems. But how does climate mechanistically affects organisms? The basic foundations to address this question were laid out by ecophysiology research. Interestingly, at the time that some of seminal research on ecophysiology was carried out, back in the 1940's (see review in (Huey 1982)), climate change was not yet in the radar of most people. The research was driven by the interest in the basic understanding of how biophysical conditions affect organisms. Today the knowledge gained from this research has acquired new importance. This is an area whether the mechanistic knowledge learnt from theory and experiments is now sometimes overlooked because of the massive datasets and machine learning approaches available. But let's start with the basics.

1.1 A model for the body temperature of an animal

Animals can be divided in two big groups in regard to the way they regulate their temperature: ecoctherms and endotherms (Figure 1.1). Endotherms such as mammals and birds are able to regulate their temperature by producing heat through metabolism. Ectotherms in contrast must regulate their temperature by obtaining heat from the environment. By obtaining energy from the environment, ectotherms have lower energy demands than endotherms, which have to be burning energy all the time to keep their bodies warm. Both do have to avoid getting too hot because indeed there can bee too much from a good thing.

To build a model of the body temperature of an animal, let's consider a lizard that is perched in a rock basking in the sun (Figure 1.2). The lizard receives heat from the direct solar radiation than can be measured in for instance calories per hour. It can also receive indirect solar radiation, reflected for instance by the ground. The lizard also exchanges heat with the surrounding air through convection. If the lizard body temperature is higher than the surrounding air temperature it will lose heat, while if the lizard body temperature is lower than it will gain heat. The rate at which this exchange of heat through convection happens depends on the body of the animal



Figure 1.1: Shrews (left, *Crocidura russula*) are ectotherms and have high energy demands, eating almost their body weight every day. In contrast, vipers (right, *Vipera seonae*) can go days without eating, or even hibernate or aestivate as needed.

and the properties of its skin. These properties are captured in the heat transfer coefficient, which can be measured as calories per hour per degree Celsius. Similarly, the lizard can receive heat by conduction from being in contact with a warm rock, or lose heat to the rock if the rock is colder than the lizard's body temperature. Finally the animal can lose heat through evapotranspiration, this is by transpiring water that has a cooling effect when it evaporates. I also like to think about this model as representing our own experience on the beach. We can be laying on a towel receiving heat by conduction from the sand and solar radiation from the sun. If the air temperature is warm, we can become uncomfortable in the sun and we may seek a shade to reduce the input from solar radiation, but if there is a cool brise we may be able to stay in the sun a bit longer. If the air temperature is really cold and it's really windy our skin hair may rise to reduce the convection coefficient.

The total flow of energy into the lizard, also known as the heat exchange equation, can be written as

$$f = q - k(b - a) \tag{1.1}$$

where

- f is the energy flow (cal/h),
- q is the quantity of heat in the solar radiation (cal/h),
- k is the convection coefficient (cal/h/^{o}C),
- b is the body temperature (^{o}C) and
- a is the air temperature (^{o}C).

If the energy flow is positive, then the lizard is warming, while if it is negative then the lizard is

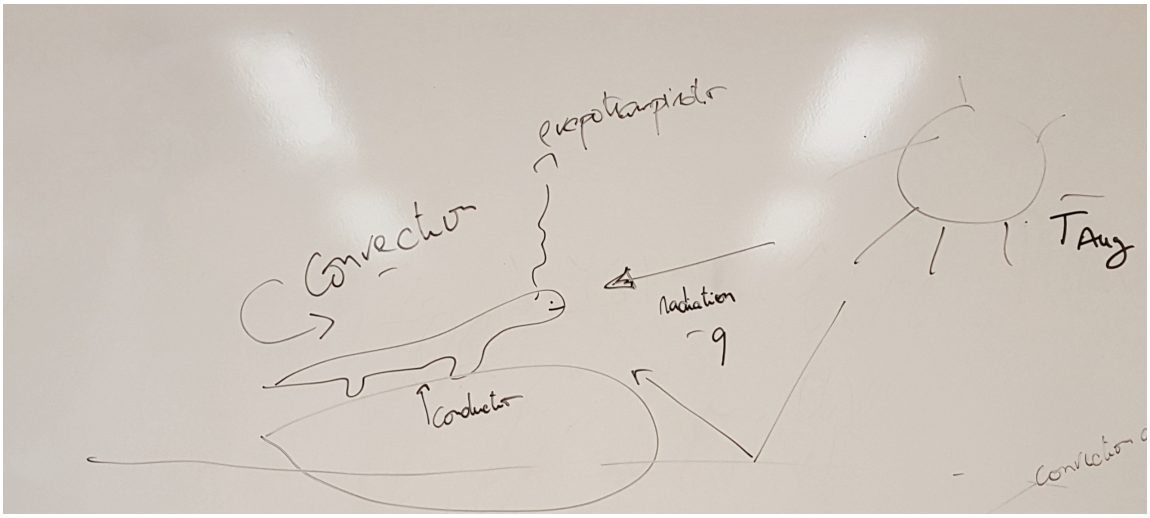


Figure 1.2: The energy flows of a lizard basking in the sun.

cooling. The equilibrium[1] happens when the energy flow is zero and so the the lizard is neither cooling nor warming. If we solve Equation 1.1 for equilibrium by replacing f with zero and expanding the right-hand side of the equation,

$$0 = q - k\ b + k\ a$$

and rearranging for b we find the equilibrium body temperature that we denote with a hat,

$$\hat{b} = q/k + a. \tag{1.2}$$

Let's see if this equation makes sense. It says that the equilibrium body temperature is the sum of the solar radiation divided by the convection coefficient with the air temperature. So, at minimum the equilibrium body temperature is equal to the air temperature when the solar radiation is zero (e.g. during the night). But when the solar radiation is greater than zero then the equilibrium body temperature is higher than the air temperature as one would expect. How much higher? Well that depends on the convection coefficient. If the convection coefficient is very high then the body temperature is mainly determined by the air temperature. In contrast, if the convection coefficient is very low then the solar radiation can contribute significantly to the body temperature.

It's time to start using **R** to explore this model. For instance, we can use **R** to plot the relationship between the body temperature and the solar radiation. First we create variables for the heat coefficient and the air temperature and assign some valuues:

[1]The concept of equilibrium is very important in ecological theory. It is often introduced in the context of differential equations and corresponds to the point at which the derivative of the variable of interest is zero. The heat equation can also be seen as a differential equation where f corresponds to the derivative of the amount of heat Q in the lizard through time t, i.e. dQ/dt.

```
k = 50  #convection coefficient (cal/h/ºC)
a = 18  #air temperature (ºC)
```

We want to plot the equilibrium body temperature for a range of solar radiation values. So we create a vector with radiation values, for instance ranging from 0 cal/h to 1500 cal/h in steps of 500. I am going to use big letters to denote vectors in **R** code, in contrast with scalars which I will denote with small letters.

```
Q = seq(0,1500,by=500)  #vector with radiation values
```

Now we can write Equation 1.2 in **R** to produce a vector of the equilibrium body temperatures for each value of radiation.

```
B_eq = Q/k+a   #vector with equilibrium body temperatures
```

Let's examine the values of the vector, by binding the two vectors as a matrix,

```
rbind(Q,B_eq)
```

```
     [,1] [,2] [,3] [,4]
Q       0  500 1000 1500
B_eq   18   28   38   48
```

This is nice as we can see the values of the equilibrium body temperature for each value of solar radiation. But let's visualize this as a graph, by plotting these vectors in **R**,

```
plot(Q,B_eq,type="l")
```

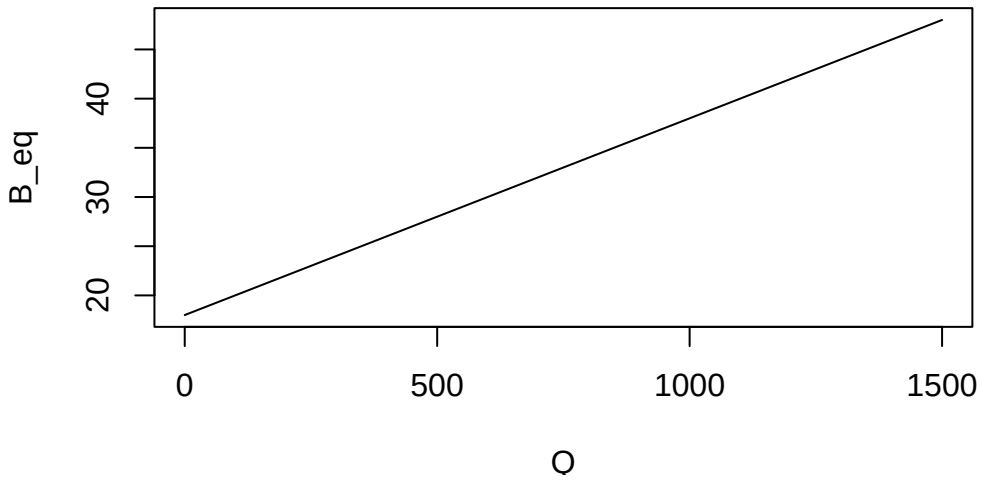


Figure 1.3: Equilibrium body temperature as a function of solar radiation.

Note that we added `type="l"` as a parameter of the plot to have a line drawn instead of a sequence of dots in the plot.

1.2 Understanding the climate space

Another way of looking at the heat exchange equation Equation 1.1 is to look at what combinations of air temperature and solar radiation values are livable for the lizard. This is also known as the climate space of an organism. In order to explore the climate space of the lizard we need to first assess what are the maximum and minimum body temperature that the lizard can experience. Let's assume those are respectively 36 and 24ºC and store them in **R,**

```
b_max = 36  #maximum body temperature (ºC)
b_min = 24  #minimum body temperature (ºC)
```

We now want to solve Equation 1.1 at equilibrium for the air temperature as a function of the solar radiation and body temperature. We can do this by rearranging Equation 1.2,

$$a = b - q/k. \tag{1.3}$$

This equation can be then used in **R** to calculate vectors of the maximum and minimum survivable air temperatures for each value of the solar radiation in vector `Q`,

```
A_max = b_max - Q/k  #vector with maximum air temperatures
A_min = b_min - Q/k  #vector with minimum air temperatures
```

We can now visualize the climate space, this is, the combination of solar radiation and air temperatures in which the lizard can survive, by plotting these two equations (i.e. by plotting lines with x coordinates given by the vector Q and the y coordinates given by the vectors A_max and A_min),

```
plot(Q, A_max,type="l", #plots the maximum survivable air temperature
     xlab="Solar radiation (cal/h)", #adds x and y axis labels to the plot
     ylab="Air temperature (ºC)")
lines(Q,A_min) #adds line for the minimum survivable air temperature
```

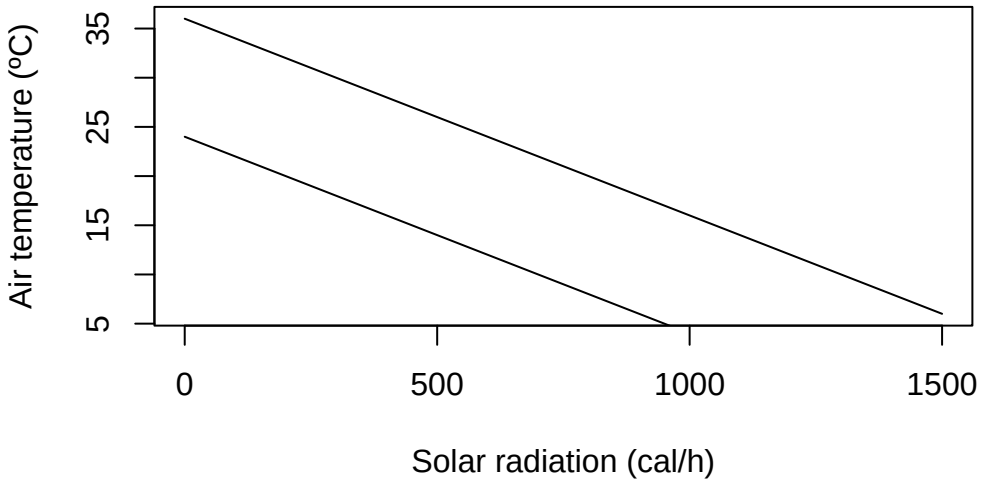


Figure 1.4: The climate space of a lizard.

Let's paint the area between the two lines which corresponds to the climate space of the lizard. We can use the **R** function `polygon`, to which we need to give the set of `x` and `y` coordinates delimiting the climate space as vectors. For instance, the upper lower corner can be the first coordinate, being 0 for the `x` vector and `b_min` for the `y` vector. The right upper corner is 1500 for `x` and `b_max-1500/k` for `y`.

```
X = c(0,0,1500,1500)
Y = c(b_min,b_max,b_max-1500/k,b_min-1500/k)
polygon(X,Y,col="green")
```

Now that we have the climate space we can plot on top of it some empirical data of how the conditions are in the field during the day. We can for instance consider two micro-habitats, a rock (in the sun) and a bush (in the shade). Suppose we obtain some data from temperature loggers that were installed in each micro-habitat , recording every three hours the values of temperature and radiation as tabled below.

Table 1.1: Hypothetical radiation values in two micro-habitats and air temperatures at different times of the day.

Time (hh:mm)	Radiation rock (cal/h)	Radiation bush (cal/h)	Temperature (ºC)
00:00	150	150	18
03:00	150	150	13
06:00	800	450	10
09:00	1100	600	14
12:00	1300	650	21
15:00	1200	650	24
18:00	800	350	22
21:00	400	200	20

We can overlay these values of temperatures and radiations on the plot to understand which micro-habitat should the lizard choose at each time of the day. First we create a vector for each column of Table 1.1. Note that we must enclose the times of the day in commas as they are strings. We also append at the end of the vector the values for 0:00 in order to close the lines (otherwise there would be a gap between 21:00 and 0:00).

```
#Times of the day
T = c("00:00","03:00","06:00","09:00",
      "12:00","15:00","18:00","21:00","00:00")

#Solar radiation in the rock habitat
Rock_q = c(150,150,800,1100,1300,1200,800,400,150)

#Air temperature in the rock habitat
Rock_a = c(18,13,10,14,21,24,22,20,18)

#Solar radiation in the bush habitat
```

```
Bush_q = c(150,150,450,600,650,650,350,200,150)

#   Air temperature in the bush habitat
Bush_a = c(18,13,10,14,21,24,22,20,18)
```

We overlay these vectors in the figure by invoking the function `lines(Xcoor,Ycoord)` for each micro-habitat. We add labels to each point to show the correspondence between each point and the time of the day.

```
lines(Rock_q,Rock_a,col="orange")
text(Rock_q,Rock_a,T)

lines(Bush_q,Bush_a,col="blue")
text(Bush_q,Bush_a,T)
```

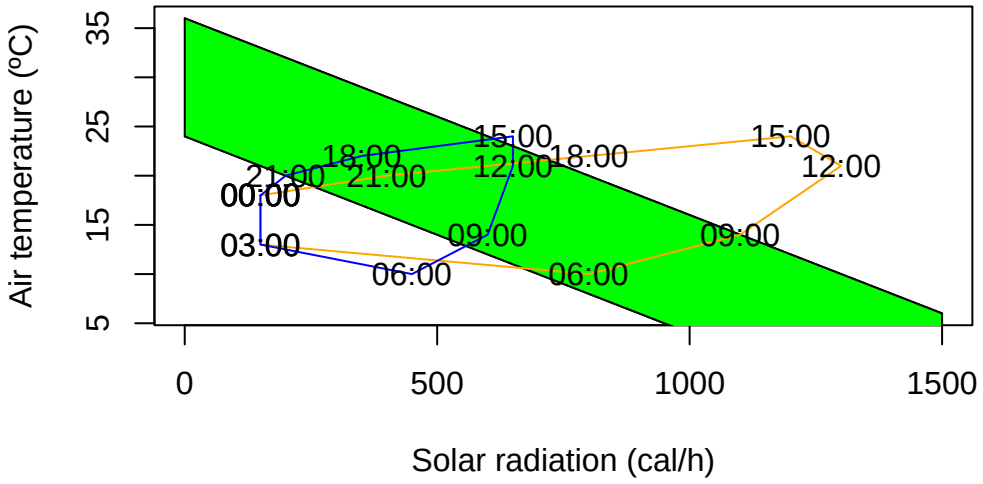


Figure 1.5: The climate space of a lizard with the conditions of two microhabitats ploted at different times of the day: rock (orange), bush (blue).

1.3 From ecophysiological models to species distribution models

In the previous section we developed a simple mechanistic model for the climate space of an organism. This is a micro-habitat level climate space. But one can also infer the climate space from the distribution of a species at the macro-habitat level. They are different but they are conceptually similar. The inference of such macro climate space is an area where species distribution models excel (for more on species distribution models check for example (Guisan, Thuiller, and Zimmermann 2017)). The idea is relatively simple. One starts with a bunch of locations of a species in geographical space. Then, one obtains the climate characteristics for those locations, and then plots those locations in climate space. The convex hull in climate space delimited by those locations is the

macro-habitat climate space. What species distribution models (SDMs) learn to do is to delineate that climate space, this is, SDMs try to predict the probability of a species occurring somewhere in the climate space by learning from presences and absences from species. Then the SDM can extrapolate again what happens in geographical space, by predicting the species occurence probability for any point based on the climate data for that point. Without getting all the way into a full blown SDM, let's try to follow some of the basics.



Figure 1.6: The Iberian emerald lizard, *Lacerta schreiberi*.

I will illustrate this with one my favorite species, the Iberian emerald lizard, *Lacerta schreiberi*, a species endemic to the Iberian peninsula, often found near water streams in mountain landscapes, but also in other habitats. First we obtain the presences for a species from GBIF (Global Biodiversity Information Facility, http://www.gbif.org). GBIF is a repository for biodiversity observations, and has over three billions of occurrences of over a million species all over the world at the time I am writing these words. Natural history museums, citizen science platforms such as iNaturalist (http://www.inaturalist.org) and eBird (http://www.ebird.org), and scientists, publish their species records in GBIF using a data format called Darwin Core. There is a R package that allows one to download observations from GBIF and also other relevant geodata, the **geodata** package. We will be doing some geospatial processing and simple SDM analysis, so let's load also the **terra** and the **predicts** packages.

```
library(geodata)
library(terra)
library(predicts)
```

We download the observations of *Lacerta schreiberi* from GBIF with the function `sp_occurence(genus,species)`. We also download a world map with the function `world(path)`, where path should point towards a local directory where we want to store the data.

```
occ <- sp_occurrence("Lacerta","schreiberi")
world_data <- world(path = ".") #download world countries limits
```

The variable occ is a dataframe with thousands of observations (rows) and several measurements/fields per observation (columns). You can inspect those attributed in the environment window in R studio or using the function `names()`. We will be mainly interested in the fields `lat` and `lon`, which store the latitude and longitude of each observation. We can now plot the map of Europe and add the occurrences on top of it using `points()`, mapping the longitude to the x-axis and the latitude to the y-axis. We crop the extent of the map to Europe, which is located between longitude 15º West and 35º East and between 35º and 72º North.

```
ext_eur <- c(-15,45,35,72) # coordinates of European extent
plot(world_data,
     xlab = "Longitude", ylab = "Latitude", axes = TRUE, ext=ext_eur)
# add the species occurrences
points(occ$lon,occ$lat,cex=.1,col="blue")
```

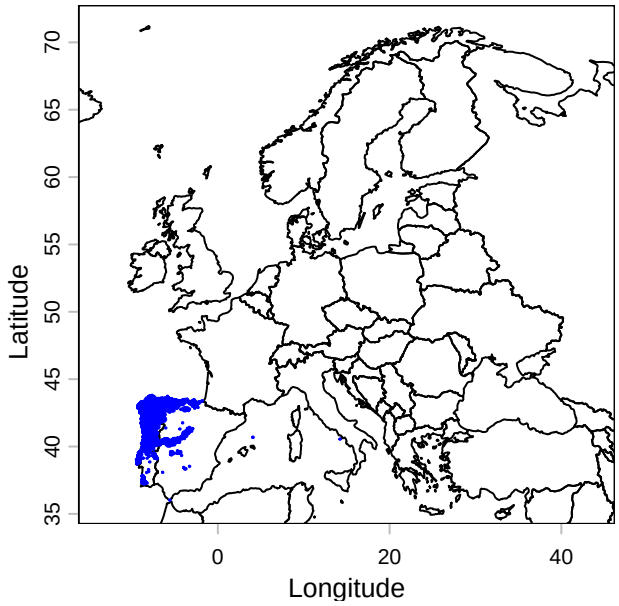


Figure 1.7: Point occurences of *Lacerta schreiberi* from GBIF.

We can see that all occurrences of *L. schreiberi* are confined to the Iberian Peninsula, particularly the Northwest of it. What is the climate space of these occurrences? There is a global dataset, the WorldClim dataset (http://www.worldclim.org), which has climate data[2] for a range of variables from minimum temperature to precipitation, monthly from 1970-2000, at a range of spatial resolutions, from 10 seconds (approximately 1 km^2) to 10 minutes (c. 18x18km^2). This dataset also includes climate normals (i.e. historical averages) for 19 bioclimatic variables, i.e. climate variables that have been found to be particularly important for modelling the distribution of biodiversity. We can download the WorldClim database using the function `worldclim_global()`, specifying the variables (`"bio"` stands for all 19 bioclimatic variables), the resolution (`10''`) and the path for the downloaded data (`"."` means the current working directory).

```
bioclim <- worldclim_global(var = 'bio', res = 10, path = ".")
```

Let's plot two of these variables, BIO3, the Isothermality which is the mean diurnal temperature range divided by annual temperature range (x 100), and BIO19, the precipitation of the coldest quarter. You can investigate what the other variables are at the WorldClim website which has a table with a brief description of all bioclimatic variables.

```
par(mfrow=c(1,2)) # prepare R to receive two plots side by side
plot(bioclim[[3]],ext=ext_eur, main = "BIO3 - Isothermality")
plot(bioclim[[19]],ext=ext_eur, main = "BIO19 -\n Precipitation coldest quarter")
```

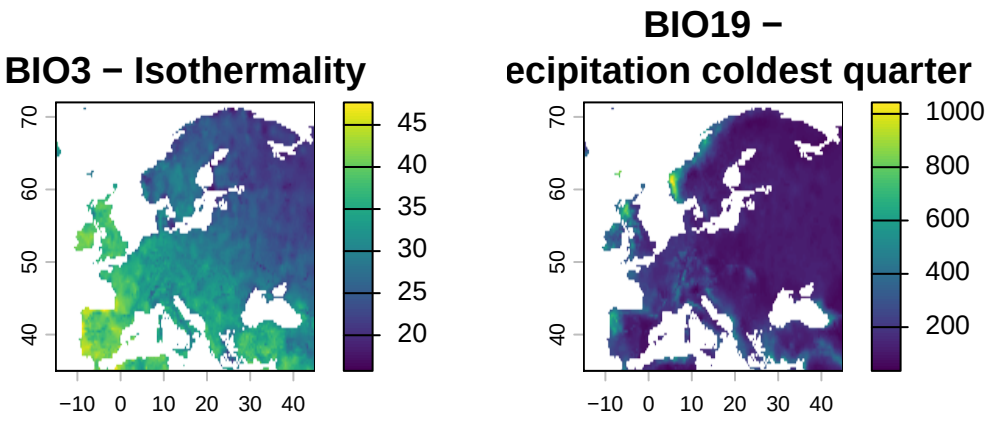


Figure 1.8: Climate variables BIO3 and BIO19 in geographic space

From a first inspection of Figure 1.8 it seems that our lizard occurs in areas with high isothermality and high precipitation on the coldest quarter of the year. Let's plot the occurrences in climate space, mapping BIO19 to the x axis and BIO3 to the y axis. First we extract the values of those climatic variables at each occurrence,

```
bioclim_occ<-extract(bioclim,cbind(occ$lon, occ$lat))
plot(bioclim_occ[,19],bioclim_occ[,3], col="blue", xlab="BIO19", ylab="BIO3")
```

[2]WorldClim is based on spatial interpolation of tens of thousands of weather stations using models and covariates such as elevation.(Fick and Hijmans 2017).

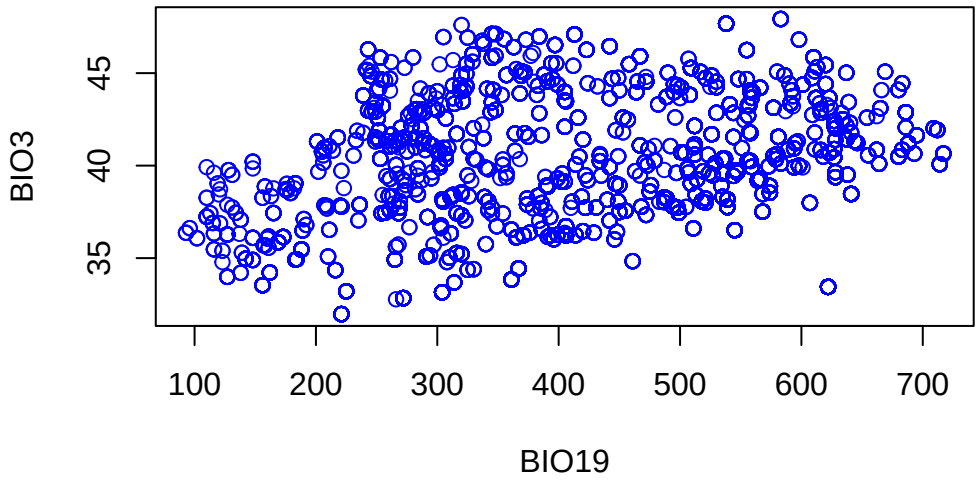


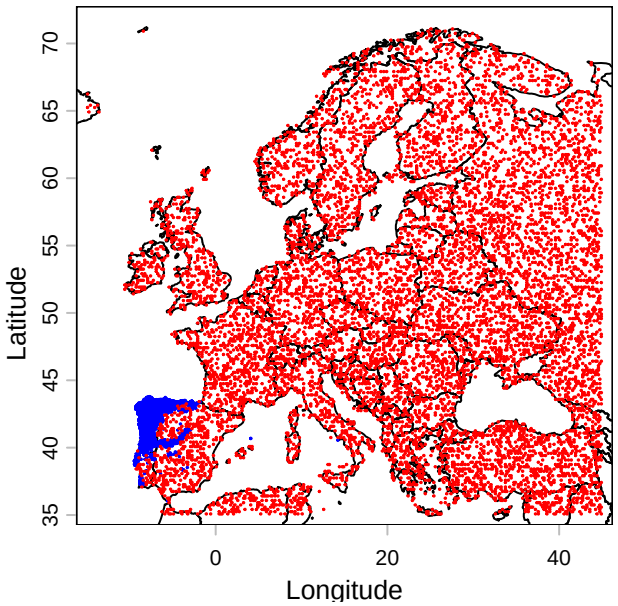
Figure 1.9: Climate space of the presences of L. schreiberi

This figure show the climate space of the presences and has some conceptual similarities with Figure 1.5. However, without mapping the climate space of the absences, it is difficult to see whether these two climatic variables really delimt the area of distribution of *Lacerta schreiberi* in climate space. The GBIF Darwin Core format was designed with museum collections in mind, and it had limited support for structured biodiversity surveys where absences and sampling effort are recorded. This has changed in the last few years with the advent of the Darwin Event extension (Wieczorek et al. 2014), but most data in GBIF is still on occurrences. Most SDMs need absences, so modellers came up with the idea of generating pseudo-absences, this is points that are randomly generated to produce a "background" signal of absences for the models to be trained on. The package **predicts** has a function backgroundSample which does this and takes a SpatRaster with grid to be sample, the number of points to be generated and a SpatVector with the occurrences that are known (so that no absences are generated in the same grid cells as the occurrences). First, in order to generate a `SpatRaster`, we crop the first bioclimatic variable to the extent of Europe, `ext_eur`.

```
eumask<-crop(bioclim[[1]], ext(ext_eur))
absences<-backgroundSample(eumask,nrow(occ),vect(occ))
```

We can now plot the presences and absences together in geographic space,

```
plot(world_data,
     xlab = "Longitude", ylab = "Latitude", axes = TRUE, ext=ext_eur)
# add the species occurrences
points(occ$lon,occ$lat,cex=.1,col="blue")
points(absences,cex=.1,col="red")
```



And now in climate space,

```
bioclim_absences<-extract(bioclim,absences)
par(pty="s")
plot(bioclim_occ[,19],bioclim_occ[,3], col="blue", xlab="BIO19", ylab="BIO3", ylim=c(0,50))
points(bioclim_absences[,c(19,3)], col="red")
```

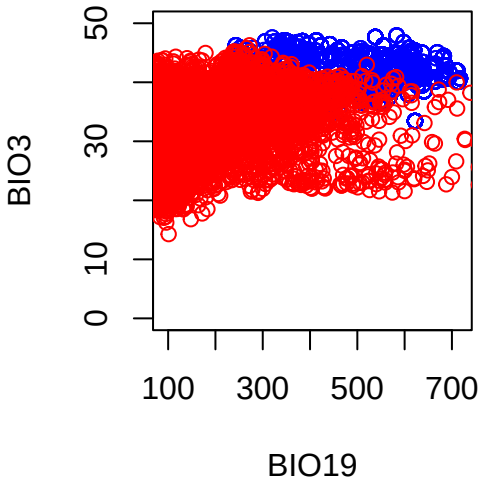


Figure 1.10: The climate space of *Lacerta schreiberi*. Presences in blue and absences in red.

It is now apparent that these two bioclimatic variables separate relatively well points in Europe where the species occur from the points where the species is (pseudo)absent, although there is some

limited overlap. This suggests that a species distribution model trained on this data would be able to separate both clouds of point and would have a good predictive power. However, we cannot tell whether this predictive power is due to chance (and spatial autocorrelation, see for instance Dormann et al (2007)) or uncovers a causal mechanism. This correlative approach contrasts with the approach in the first sections of this chapter where we tried to develop a mechanistic model from first principles.

1.4 Statistical confrontation: regressing the climate space

By Andres Marmol

Why anoles? Anoles are among the most studied reptile organisms. They are vastly diverse with more than 488 species described until today — circa 11% of all the squamates without considering snakes — due to their outstanding adaptive radiation, which has inspired many to dive-in-depth into understanding how evolution has operated within this group (Lizards in an Evolutionary Tree by J. B. Losos (2009) is a highly recommended instructive reading for more info). As a result, very detailed data sets on many species of anoles, including information on micro habitat, field body temperature, activity patterns and morphological traits, performance, etc., can be found online in relatively few time.

In the following steps we will be downloading from Dryad and using a data set created by Winchell et al. (2016) on *Anolis cristatellus* making a pair-comparison of several morphological and thermal traits of individuals living in urban and forested areas at Puerto Rico. *A. cristatellus* is widely distributed across Puerto Rico and can be found in natural and human-intervened spaces easily. It can be seen over ground and laying on the trunks of trees in natural settings, and on walls and metal fences in more urban areas.

1.4.1 Calling the dataset from Dryad

To download the dataset, it is necessary to install the R package `rdryad`.

```
install.packages("rdryad")
```

Then we open the newly installed package and download the dataset by specifying the DOI associated to this dataset.

```
#calling rdryad
library(rdryad)

# Specify the DOI
doi <- "10.5061/dryad.h234n"

# Download the dataset using the dryad_download function
downloaded_file <- dryad_download(dois = doi)
```

The data sets are now downloaded on the paths indicated in the box below, although for this exercise only table [2] winchell_evol_phenshifts.csv" is needed.

```
# The downloaded_file will contain the file path where the dataset is saved
print(downloaded_file)
```

```
$`10.5061/dryad.h234n`
[1] "/Users/henrique/Library/Caches/R/rdryad/10_5061_dryad_h234n/winchell_evol_CG.csv"
[2] "/Users/henrique/Library/Caches/R/rdryad/10_5061_dryad_h234n/winchell_evol_phenshifts.csv"
```

Once this is done, it is always good practice to check if the recently open data frame works by checking the columns within the data frame.

```
#calling the dataset of interest as df
df <- read.csv(downloaded_file[[1]][2])

# checking the columns (variables) available in the data frame df using the str function
names(df)
```

```
 [1] "ID"                "date"              "Site"
 [4] "context"           "perch"             "bodytemp.C"
 [7] "perch.temp.C"      "ambient.temp.C"    "humidity.percent"
[10] "perch.height.cm"   "perch.diam.cm"     "weight.g"
[13] "head.height.mm"    "svl.mm"            "local.time.decimal"
[16] "flags"             "JL"                "JW"
[19] "METC"              "RAD"               "ULN"
[22] "HUM"               "FEM"               "TIB"
[25] "FIB"               "METT1"             "METT2"
[28] "FL"                "HL"
```

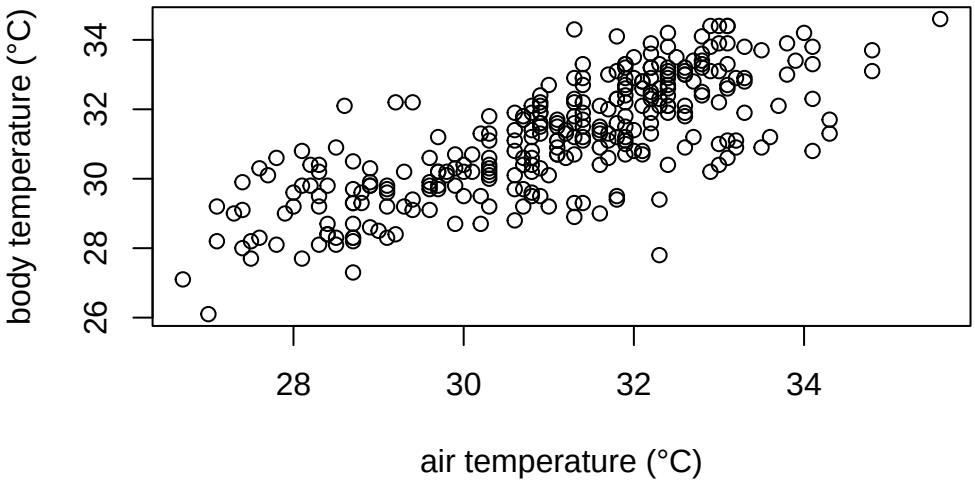
There are 29 different variables within `df`, but this exercise will focus only in a few of them including `context`, `perch`, `bodytemp.C`, `perch.temp.C`, `ambient.temp.C`, and `local.time.decimal` where `context` indicates if the lizard was captured in natural or urban spaces; `perch` refers to the microhabitat where the lizard was found; `bodytemp.C` is body temperature b; `perch.temp.C` is substrate temperature T_s; `ambient.temp.C` is air temperature a; and `local.time.decimal` is the time of the day in hours when the lizards was captured and measured.

1.4.2 Exploring the thermophysiological data of *A. cristatellus.*

As covered in the first chapter, b increases proportionally to a as indicated in equation (1.2). Let's check if the data confirms this equation.

```
# to call a variable directly from a data frame one must use $, indicating the name of the dataframe

plot(df$ambient.temp.C, df$bodytemp.C,
     xlab="air temperature (°C)",
     ylab="body temperature (°C)")
```



In the plot we can observe that indeed body temperature increases with increasing temperature. However, we cannot quantitatively know how much is this increase. For that, we can model how air temperature affects body temperature modelling them a a line with the equation:

$$\hat{y} = \alpha + \beta * x + e$$

where $\hat{y}$ is the predicted value of y for any given x; α is point of intersection of the line when $x = 0$, β is the slope of the regression line (i.e. how much increases $\hat{y}$ with a unit increase in x) and e is the random error.

Placing our variables in formula, it would look like this:

$$b = b_0 + \beta * a + e$$

where b is body temperature, a is ambient temperature, and b_0 is the body temperature when a is zero.

Now lets model it using a least-squares linear regression using the `lm` function in R.

```
#define the linear model
#the variable at the left of the tilde (~) is the dependent variable
#data calls the specific dataframe to be used.

model1 <- lm(bodytemp.C ~ ambient.temp.C, data=df)

# print model outputs
print(model1)
```

```
Call:
lm(formula = bodytemp.C ~ ambient.temp.C, data = df)
```

```
Coefficients:
   (Intercept)  ambient.temp.C
        8.7004          0.7237
```

1.4.3 Interpreting the results of our model.

The results of our model indicates that if the air temperature (a) is zero, *A. cristatellus* would show a body temperature (b) of 8.7 °C, which increases in approximately 0.7 °C for each 1 °C increase in air temperature.

Now it is also possible to draw a few predictions

```
# a number of possible ambient temperatures that can be found in Puerto Rico during the day

amb_temps = data.frame(ambient.temp.C = seq(27,38, 1))# defines a vector with values from 27 to 38,

predict(model1, newdata=amb_temps)
```

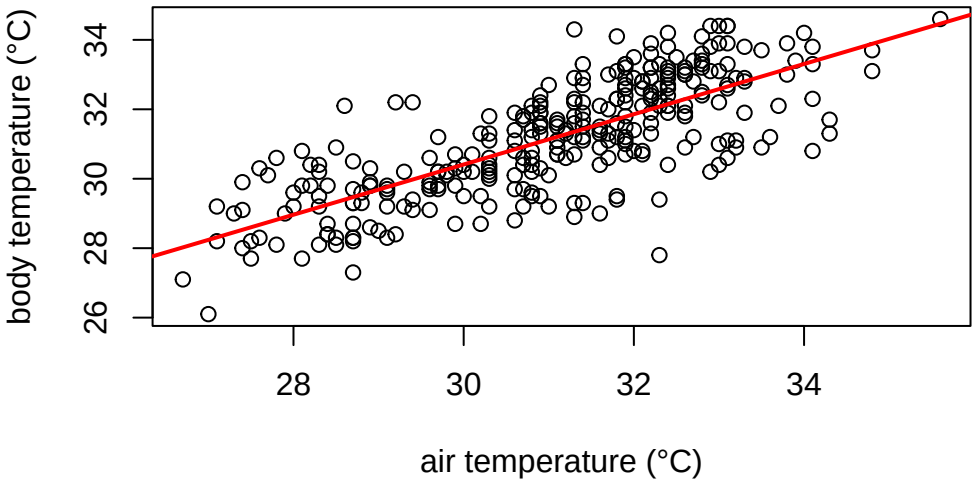
```
       1        2        3        4        5        6        7        8
28.23967 28.96335 29.68702 30.41070 31.13437 31.85805 32.58172 33.30540
       9       10       11       12
34.02907 34.75275 35.47642 36.20010
```

1.4.4 How good our model is performing?

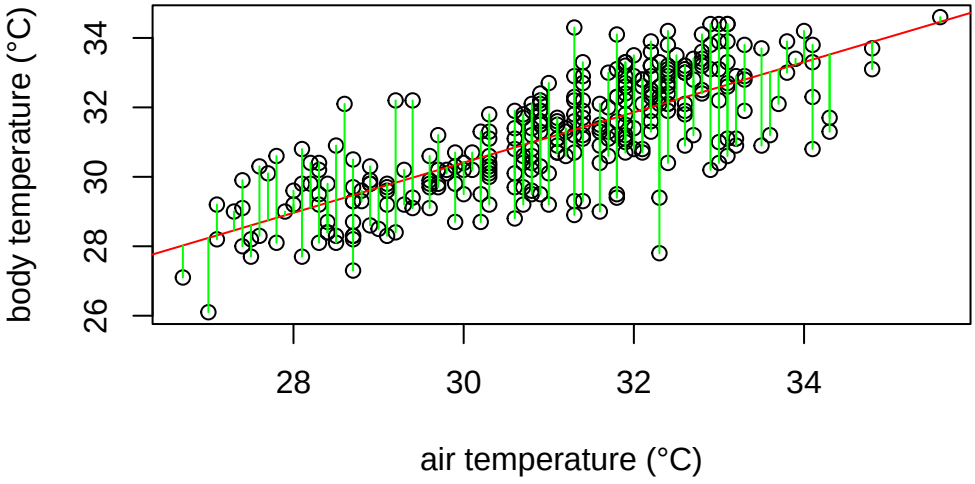
Graphically the linear model fits the data as follows:

```
# Create the scatter plot
plot(df$ambient.temp.C, df$bodytemp.C,
     xlab="air temperature (°C)",
     ylab="body temperature (°C)")

# add the line of fit from our recently created linear model "model1"
abline(model1, col = "red", lwd= 2)
```



The red line in the plot follows very well the pattern of increase shown by our data. It is noticeable, however, that some points fall farther from the line of best fit than others. In other words, many $\hat{y}$ values (body temperature) are predicted with larger errors than others, for a given x value (ambient temperature). These errors are also known as residuals, and are the vertical distance between the observed values and the line of best fit. They are graphically represented in green in our plot:



What a least square linear regression does is to find the lowest sum of squares of the residuals (every green line in the plot) so that the least amount of error is kept in the linear model.

Finally, how well the model fits the data? In our example, the body temperatures do not perfectly fall on the line of best fit, and many of them are scattered around it. A common metric assess this question in a simple regression model is the r^2. r^2 values indicates how much of the variance of body temperature (our dependent variable) is explained by ambient temperature (our independent variable). A perfect fit is indicated by an r^2 of 1, while no fit would be zero. To check the r^2 in our model we can simply use the `summary` function in R.

```
# obtaining the r-squared of our model

summary(model1)$r.squared
```

```
[1] 0.5736088
```

r^2 equals 0.574 indicating that almost 60% of the variance in body temperature is explained by ambient temperature.

In summary, our expectation that lizards body temperature increases depending on how the temperature in their environment changes has been confirmed. Specifically for *A. cristatellus* in this study, body temperature increases ~ 0.7 °C every 1 degree increase in air temperature. A few questions remain. For example, does habitat has an effect in the body temperature in *A. cristatellus*, and if it does, how different could the body temperatures of the lizards in habitat A are from those in habitat B?

Chapter 2

Economic models of behavior: do animals optimize?

This chapter is under construction. Content may change.

Animals have to make decisions all the time: where and when to move, when and what to eat, whom to mate, etc. How do animals make those decisions? One approach to study this question is to assume that animals take decisions that maximize their fitness. This approach is rooted in the idea that natural selection, under certain conditions, maximize the average fitness of individuals in a population. It underpins many of the studies in behavioral ecology, and lead to the development during the 1970's and 1980's of a sub-field dedicated to the study of foraging decisions, known as optimal foraging (see for instance Pyke 1984). The idea of optimal foraging is to develop economic models of foraging behavior, assessing the costs and benefits of different actions, and identifying in any particular circumstance the action that maximizes benefits and minimizes costs. This is, animals are seen as optimizer of decision-making. Here we consider models for two "idealized" types of foragers: searching predator and sit-and-wait predators.

2.1 Searching predator

The searching predator is typified as an animal that actively searchs for food. For instance, wolfs roam the landscape constantly searching for their prey (Figure 2.1). The question is then, any time that a forager comes across a prey whether to spend the time catching and eating it, or continuing to search for another prey, for instance a larger or easier prey item. Joan Roughgarden also liked this type of foraging to the sushi-bar problem where dishes are coming one after the other on a moving tray, and a person needs to decide whether to pick a given dish. This decision is particularly akin to the searching predator situation when one assumes that a person can have only one dish at a time in her or his table, We consider two types of searching predators: time minimizers and energy maximizers.



Figure 2.1: A pack of wolfs (*Canis lupus*) roaming around the landscape of Peneda-Gerês National Park, Portugal.

2.1.1 Time minimizer

Many searching predators are themselves preys to other animals. For instance shrews search for insects to eat, but can be themselves eaten by carnivores or owls. Therefore a reasonable assumption is that they are making foraging decisions that minimize the time foraging. Let's assume there are two types of prey items, type 1 and 2. These two preys occur at different abundances in the environment, let's name them a_1 and a_2. These abundances can be measured as encounter rates from the predator perspective, this is the number of prey items found per unit time. The two types of prey also have different handling times, this is the amount of time required to chase and process the prey, h_1 and h_2 . We also convention to call the type 1 prey the prey with the lowest handling time, i.e. $h_1 < h_2$,

A searching predator can adopt one of the following three strategies:

- Strategy 1: to consume only prey items of type 1
- Strategy 2: to consumer only prey items of type 2
- Strategy 1&2: to consumer both types of prey

In order to find out which strategy should be adopted by the forager, one needs to calculate the average spent per food item in each of the strategies. For Strategy 1 the average time per item, T_1 is the sum of the amount of time the predator needs to encounter a prey with the amount of time that it takes to process that prey. As the abundance is measured in encounter rates, i.e. prey items

per unit time, the inverse of that is the waiting time for a prey. Therefore we have for Strategy 1,

$$T_1 = 1/a_1 + h_1 \tag{2.1}$$

Similarly, for Strategy 2, the average time per item T_2 is given by

$$T_2 = 1/a_2 + h_2. \tag{2.2}$$

A more interesting case is Strategy 3. Here the waiting time is the inverse of the sum of the abundances of both times of prey, while the handling time is average of the handling times of both ypes of prey weighted by their relative abundance,

$$T_3 = \frac{1}{a_1 + a_2} + \frac{a_1 h_1 + a_2 h_2}{a_1 + a_2} \tag{2.3}$$

We can now define the problem that the forager has to solve as to find the strategy i that minimizes T_i:

$$min(T_i) \quad \text{for} \quad i = 1, 2, 3$$

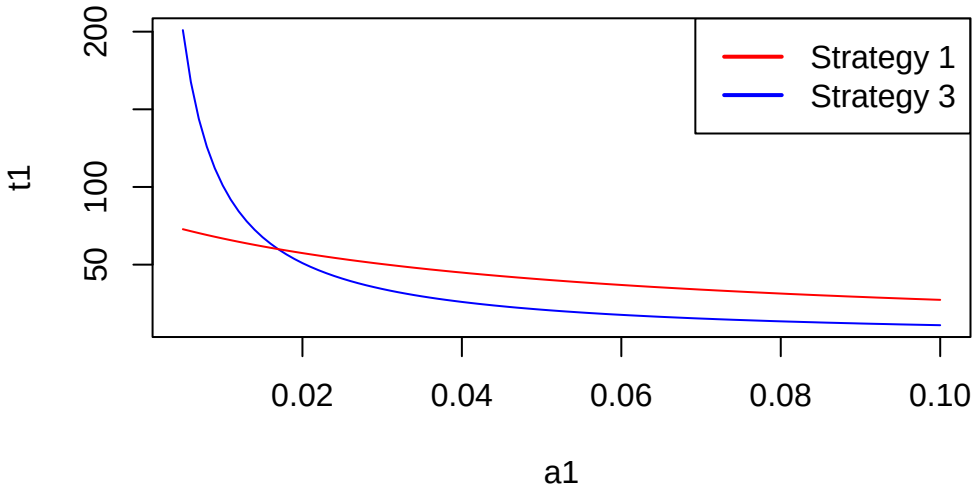
It is easy to demonstrate that Strategy 2 is always worse than Strategy 3, independently of the abundances of the two types of prey. This happens because Strategy 3 always implies a longer waiting time than the two other specialized strategies (i.e. one has to wait less time to find any item of an prey than items of a given prey time, $1/(a_1 + a_2) < 1/a_2$) and the handling time of strategy 3 can never be higher than the handling time of strategy 2 (it's always a value betwen h_1 and h_2). So we can exclude Strategy 2 from our analysis. The choice is then between Strategy 1, just taking items of the preferred prey item, and 3, taking items of both types of prey. Let's assess this two strategies with a little bit of help from R. We start by assuming that the handling time of prey type 1 is 1 second while prey type 2 takes 60 seconds. Let's also assume that the abundance of prey type 2 is 0.05 individuals per second, i.e. one individuals needs to wait in average 20 seconds to find prey type 2.

```
h1=1            #Handling time of prey type 1 (s)
h2=60           #Handling time of prey type 2 (s)
a2=0.05         #abundance of prey type 2 (ind/s)
```

Let's now plot the time per item of each of the strategies as a function of the abundance of the preferred prey.

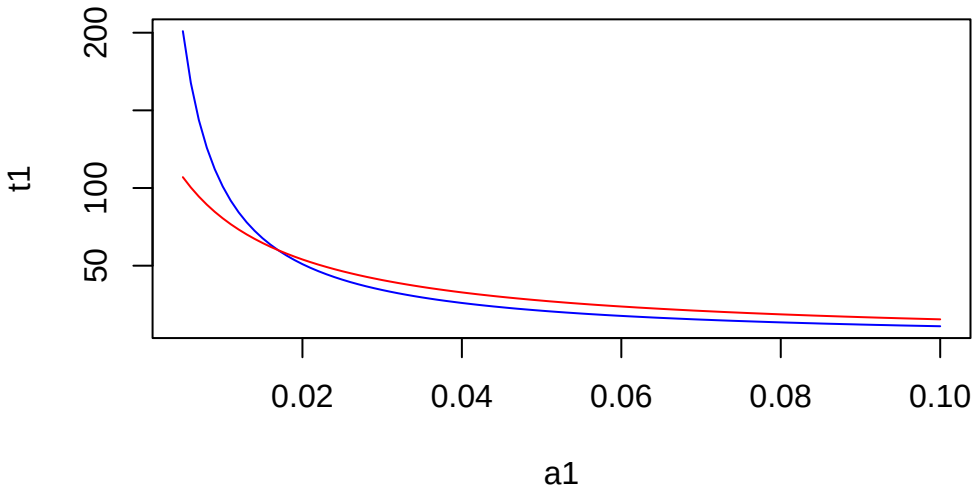
```
a1<-seq(0.005,0.1,0.001)   #abundance of prey time 1 (ind/s)
t1=1/a1+h1                 #time per item of Strategy 1 (s)
t3=1/(a1+a2)+h1*a1/(a1+a2)+h2*a2/(a1+a2) #time per item of Strategy 2 (s)
plot(a1,t1, type="l", col="blue")
lines(a1,t3, type="l", col="red")
legend("topright",
       legend = c("Strategy 1", "Strategy 3"),  # Labels
```

```
       col = c("red", "blue"),              # Line colors
       lwd = 2,                             # Line width
       lty = 1)
```



There is a critical threshold of the abundance of prey type 1 above which strategy 1 is preferrable, while below that threshold strategy 3 is the best strategy. Interestingly this threshold does not depend on the abundance of the less preferable prey. For instance, if we assume a low abundance of prey time 2 at 0.01, the resulting plot is:

```
a2 = 0.01
t1=1/a1+h1                  #time per item of Strategy 1 (s)
t3=1/(a1+a2)+h1*a1/(a1+a2)+h2*a2/(a1+a2) #time per item of Strategy 2 (s)
plot(a1,t1, type="l", col="blue")
lines(a1,t3, type="l", col="red")
```



To determine this critical threshold one can compare the two vectors, T1 and T3, and find the first position at which T1 becomes smaller than T3,

```
pos=which(t1<t3)[1]
a1[pos]
```

```
[1] 0.017
```

So the critical threshold for these handling times occurs when $a_1 = 0.017$ individuals per second.

2.1.2 Energy per time maximizer

Perhaps more often, animals try to maximize their energy yield (benefits) while minimizing the time foraging (costs). Or in another way of looking at it, they try to maximize their energy yield per unit time. We already know the time per item associated to each of the three strategies of the searching predator. We now need to caiculate the average energy yield per item. Consider now that the energy content of the prey items are e_1 and e_2 for prey of type 1 and 2, respectively. We define prey 1 as the preferred type of prey, so we assume that the ratio of the energic content (measured for instance in calories) to the handling time is higher for type 1 prey, i.e. $e_1/h_1 > e_2/h_2$. Now we calculate the energy yield per item for each strategy. We start with the energy content of the prey, but need to subtract the energy spent while waiting the prey and the energy spent chasing and processing the prey,

$$E_1 = e_1 - ew * tw_1 - eh * h_1 \tag{2.4}$$

where ew and eh are the energy spent per unit time while waiting for the prey and the energy spent per unit time while handling, respectively. They can both be measured for instance in cal/s. We already know from the time minimizer that the waiting time for the prey is the inverse of the abundance, $tw_1 = 1/a_1$. Therefore substituting in Equation 2.4 we have

$$E_1 = e_1 - \frac{ew}{a_1} - eh * h_1. \tag{2.5}$$

A similar expression can be written for Strategy 2, replacing 1 with 2 in Equation 2.4.

$$E_2 = e_2 - \frac{ew}{a_2} - eh * h_2. \tag{2.6}$$

More interesting is to derive the expression for Strategy 3, where the foragers takes both types of prey. The energy content of the prey is the average of the energetic contents of each prey type, weighted by their abundances, $(e_1 * a_1 + e_2 * a_2)/(a_1 + a_2)$. The waiting time is the inverse of the sums of the abundances of preys of both types, as in Equation 2.3. The handling time is the average of the handling times of each prey type, weighted by their abundances. So we have,

$$E_3 = \frac{e_1 * a_1 + e_2 * a_2}{a_1 + a_2} - \frac{ew}{a_1 + a_2} - eh\frac{h_1 * a_1 + h_2 * a_2}{a_1 + a_2}. \tag{2.7}$$

Finally we can calculate the energy per unit time for each strategy by dividing Equation 2.5 by Equation 2.1 for Strategy 1, dividing Equation 2.6 by Equation 2.2 for Strategy 2, and dividing Equation 2.7 by Equation 2.3 for strategy 3,

$$ET_i = \frac{E_i}{T_i}$$

Similarly to the time minimizer, it's possible to show mathematically that strategy 2 is never an optimal strategy. So the interesting comparison is again between strategy 1 and strategy 3. Let's use R to plot the energy per time yield for both strategies. We start by setting the parameter values of our model with some realistic numbers.

```
e1<-10                          #Caloric content of prey 1
e2<-100                         #Caloric content of prey 2
h1<-1                           #Handling time of prey 1
h2<-60                          #Handling time of prey 2
ew<-1                           #Energy spend per unit time while waiting (cal/s)
eh<-1                           #energy spend per unit time handling the prey (cal/s)
```

We will examine the energy yields for strategy 1 and strategy 3 for a fixed abundance of prey type 2 and a sequence of abundances of prey type 1 from 0.005 individuals per second to 0.5 individuals per second

```
a1<- seq(0.005,0.5,0.001)     #Sequence of abundances of prey 1
a2<-0.05
```

With the parameter and abundance values defined, we can calculate E_1, E_3, T_1, T_3, and then ET_1 and ET_3 based on the equations above, resulting in vectors for these variables with each entry in the vector corresponding to an abundance value in the vector of abundances `a1`.

```
E1 = e1 -(eh*h1) - ew/a1    #energy per item of Strategy 1 (s)
E3 = (e1*a1+e2*a2)/(a1+a2)-
     eh*(h1*a1+h2*a2)/(a1+a2)-ew/(a1+a2) #energy per item of Strategy 3 (s)

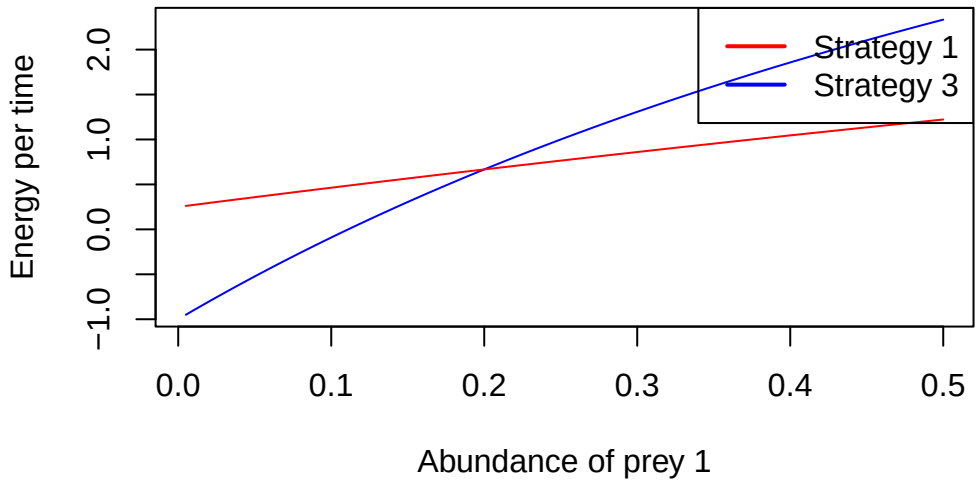
T1=1/a1+h1                  #time per item of Strategy 1 (s)
T3=1/(a1+a2)+h1*a1/(a1+a2)+h2*a2/(a1+a2) #time per item of Strategy 3 (s)

ET1 = E1/T1 #energy per time of strategy 1
ET3 = E3/T3 #energy per time of strategy 3
```

Next we plot the energy yields against the values of abundance of prey of type 1, and add a nice legend:

```
plot(a1,ET1,type="l",xlab="Abundance of prey 1", ylab="Energy per time", col="blue")
lines(a1,ET3,type="l",col="red")
legend("topright",
       legend = c("Strategy 1", "Strategy 3"),  # Labels
       col = c("red", "blue"),              # Line colors
```

```
       lwd = 2,                              # Line width
       lty = 1)
```



Similarly to the the time minimizer, for the energy maximizer there is also a critical threshold of the abundance of prey type 1 above which strategy 1 is preferrable, while below that threshold strategy 3 is the best strategy.

2.2 Sit-and-wait predator

In contrast to the searching predator, the sit-and-wait predator forages by patiently ambushing its prey. For instance, the lizard *Anolis gingivinus* (Figure 2.2) waits in a perch, often a tree trunk, for a prey to come into its reach, sprinting then down the trunk or into the ground to capture its prey, returning then to its perch again. So here the decision that the predator has to take after seeing the prey is whether it should run an capture the prey or if it should ignore it.

We will build on the model we developed for the energy per time maximizer searching predator to develop a model for the decisions of the sit-and-wait predator. The energy per prey item consumed is,

$$E = e - e_p t_p - e_w t_w \tag{2.8}$$

where e_p is the energy per unit time while pursuing the prey, t_p is the average time it takes to sprint to the prey and come back to the perch, e_w is the energy per unit time while waiting for the prey, and t_w is the average time it takes to wait for a prey item to show up within the home-range or territory of the predator. We will use these two terms interchangeably, although they are sometimes used in the literature differently.

Let's consider that the home-range of the lizard is shaped as a semi-circle centered in the perch. This is the lizard has a viewing angle of 180 deg from its perch (Figure 2.3). Then the total abundance of the the prey in the territory is the integral of the prey point density over the territory. So, using polar coordinates for the integral this can be written as,



Figure 2.2: An Anolis gingivinus, a sit-and-wait predator, captures a prey after running down from its perch. Photo taken in the island of Saint Martin, Leward Islands, Caribbean.

$$A = \int_0^{r_c} a\pi r dr \tag{2.9}$$

This is an easy integral, but let's see how we could solve it in R, as this will be handy for the next integral. We can do symbolic math in R using the package Ryacas. Ryacas provides an interface to Yacas, Yet Another Computer Algebra System. We use ysim() to tell Yacas to integrate the product $a\ \pi\ r$ from 0 to r_c with the function integrate(),

```
library(Ryacas)
A=ysym("Integrate(r, 0, rc) a * Pi * r")
```

Often we would like to see mathematical equations beautifully typeset. This can be done using TeX, and Ryacas does provide a way for layout the equation as a TeX command. However, for Quarto to recognize a TeX command, we need to start it and close it with a "$$". As we will be doing this a lot, I wrote a little function, tex_quarto, that does this and also takes an optional prefix that can be put in the beginning (e.g. the variable name and an identity sign).

```
tex_quarto = function (x, prefix=NULL)
  cat("$$", prefix, tex(x), "$$")
```

Now, we can output the result of the integral from Equation 2.9 in a beautiful way as[1],

```
tex_quarto(A, "A=")
```

$$A = \frac{rc^2 a\pi}{2}$$

The waiting time is just the inverse of the total abundance,

$$t_w = 1/A$$

or in R code,

```
tw=1/A
```

The average pursuit time is a bit more complicated. For a prey landing at distance r from the predator, the pursuit time is the time the lizards needs to run to that location and back. This is can be calculated by dividing the double of the distance by the speed of the lizard, $2r/v$. We have now to average these sprint times over the range of possible distance that the lizard can run, from 0 to the radius of its territory r_c, weighted by the probability of a prey appearing at each distance. The probability of a prey appearing at distance r is the length of the semi-circle with radius r multiplied by the point density of prey divided by the total density of prey in the territory of the lizard, $a\pi r/A$. So we now need to calculate the integral of the sprint times at each distance times the probability of running to a prey at that distance over the range of possible radius values,

[1] Note that this only displays the typeset equation when you render the Quarto document into html or PDF, and it is not shown directly on the notebook output mode. You also need to add the command "#| output: asis" in the beginning of the R chunk to tell Quarto it needs to take the output literally.

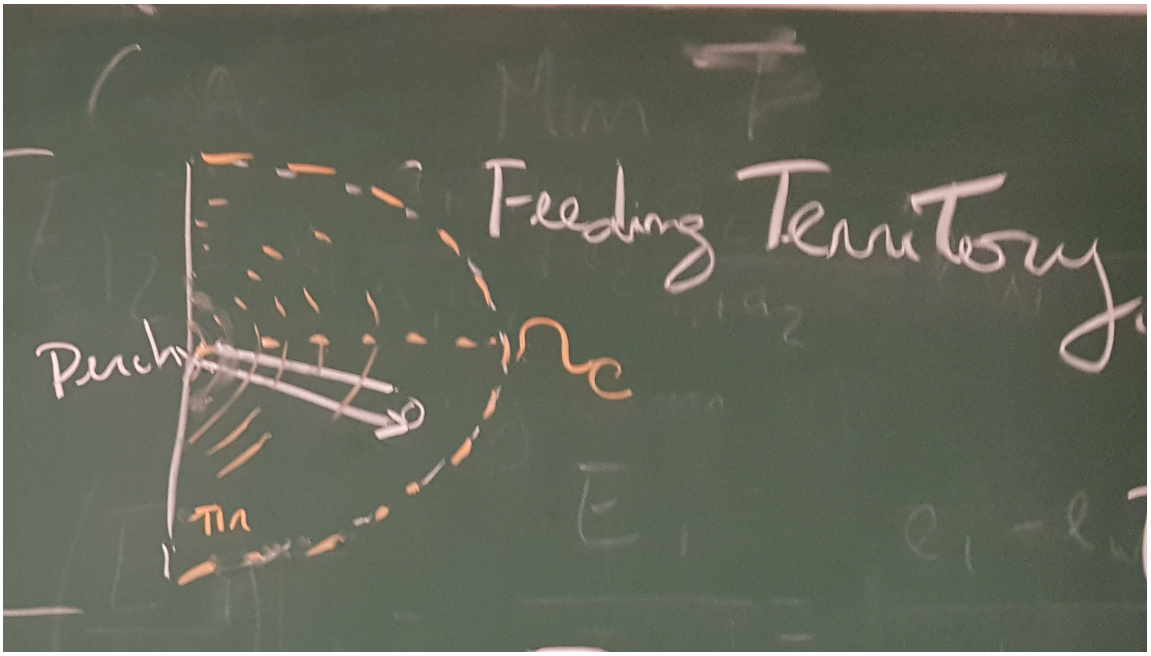


Figure 2.3: Feeding territory of a sit-and-wait predator with radius r_c.

$$t_p = \int_0^{r_c} \frac{2r}{v} \frac{a\pi r}{A} \, dr$$

Let's calculate this more complicated integral in R, substituting the value of the total abundance that we calculated above using the function with_value(), and simplifying the result with simplify(),

```
tp=ysym("Integrate(r, 0, rc) (2 * r / v) * (a * Pi * r) / A")
tp = with_value(tp, ysym("A"), A) |> simplify()
tex_quarto(tp, "t_p=")
```

$$t_p = \frac{4rc}{3v}$$

The average time per prey item consumed is simply the sum of the average pursuit time, t_p, and the average waiting time t_w,

$$T = t_p + t_w \tag{2.10}$$

Therefore the energy per time that the optimal sit-and-wait predator wants to maximize is obtained by dividing Equation 2.8 by Equation 2.10,

$$ET(r_c) = \frac{e - e_p t_p(r_c) - e_w t_w(r_c)}{t_p(r_c) + t_w(r_c)}$$

where we highlighted that the wating time and the pursuit time are functions of the territory size or cut-off radius r_c. Let's calculate this expression in R,

```
ET=ysym("(e-ep*tp-ew*tw)/(tp+tw)")
ET=with_value(ET,ysym("tp"),tp)
ET=with_value(ET,ysym("tw"),tw)
simplify(ET)
```

```
y: (3*e*rc^2*v*a*Pi-4*ep*rc^3*a*Pi-6*v*ew)/(2*(2*rc^3*a*Pi+3*v))
```

Let's plot the energy per time as a function of the territory size. We have to start by giving some values to the different paramerters.

```
ew=0.1
ep=1
v=0.5
a=0.005
e=10
```

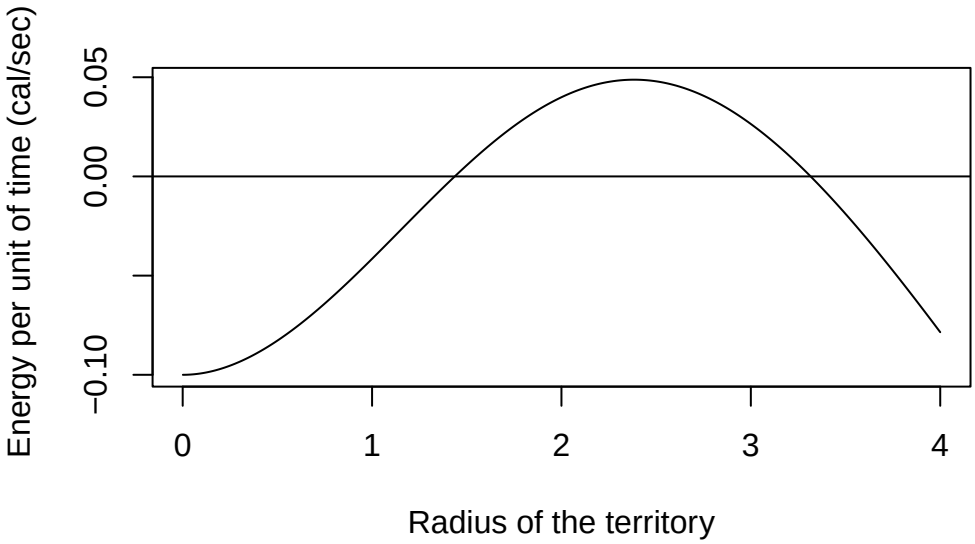
Next we convert the yacas expresssion for ET into a R expression using `yac_expr()` and then write a function that evaluates that R expression for a given value of rc,

```
ET_expr = yac_expr(ET)

ETfun <- function(rc)
  eval(ET_expr)
```

We can now produce the plot by creating a vector with a range of values for r_c and then plotting the function `ETfun`,

```
rc<-seq(0.001,4,0.001)   #Radius of the hunting ground
plot(rc,ETfun(rc),type="l",
     xlab="Radius of the territory",
     ylab="Energy per unit of time (cal/sec) ")
 abline(0,0)
```



This plot shows that the energy per time, ET is a uni-modal function of the territory size r_c. At zero radius, the energy per time is negative and equals the energy per unit of time while waiting, i.e. there are only costs and no benefits of the territory. As the territory size increases, the energy per time starts to increase and becomes positive, reaching a maximum value at a bit more than 2m for the parameters above. Then, as the costs of running very far to chase prey start exceeding the benefits, the energy per unit time declines again.

The optimal decision is to find the cut-off radius r_c that maximizes the energy per time,

$$\max_{r_c}(ET)$$

We can find the analytical solution of the maximum by differentiating **?@eq-maxet** in order to the cut-off radius and equalizing it to zero,

$$\frac{\mathrm{d}ET}{\mathrm{d}r_c} = 0$$

This means that we try to find the place in which the tangent to the function has slope zero, i.e. it is horizontal. This can be the maximum or the minimum of a function, and often one has to investigate further by looking at the sign of the second derivative, but not in this case. So, let's solve this in R,

```
dET = deriv(ET,"rc")
rcopt=solve(dET, "rc")
simplify(rcopt)
```

```
{rc-(((-243*((-3)*e*v^3*a^6*Pi^6)^2*e*v^4*a^5*Pi^5)/(54*((-3)*e*v^3*a^6*Pi^6)^3)-Sqrt(((-3*e*v^3*a^6
```

If you wanna read about an empirical test of this model, I suggest the paper by Shaffir and Rougharden (1998).

2.3 Does natural selection maximizes fitness?

Animals do not solve mathematical equations, at least in the way we do. So, how can animals find this optimal home-range sizes or choose the optimal strategy? One way of doing this is by learning, and there are simple models of reinforcement learning that can produce decisions that are optimal (see for instance Roughgarden (1998b)). Another way is through natural selection and evolutionary processes. The importance of evolution in shaping animal behavior was at the core of the research of Karl von Frisch, Konrad Lorenz and Nikolaas Tinbergen, who received the Nobel Prize in Physiology or Medicine in 1973. These three scientists played a key role in developing the science of ethology, the comparative study of behavior across species and ecological contexts. Until their work, studies of behavior were focusing on the dichotomy between stimulus-response behavior or the idea of the idea of learning as an explanation to all behavioral variations. Lorenz and Tinbergen's work showed that fixed action patterns observed in many animals were the response to particular stimuli and were done without any previous learning. Lorenz and Timbergen often worked with birds, but my favorite fixed action patterns are the pushup displays of *Anolis* in response to predators or the presence of conspesifics. von Frish discovered the "bee language", this is, how foraging bees communicate to nestmates upon their return to the nest the location of nectar an pollen resources.

So the idea that some behaviors are genetically "programmed" allows us to model the dynamics of behavior acquisition in a population of individuals, and explore the question of whether natural selection maximizes fitness. We consider a very simple model of one genetic loci with two alleles in [for a diplod model see for instance Roughgarden et al]

- Simple models of population genetics for frequency independent selection.
- main theorem of natural selection and climbing the fitness landscape or Fisher's fundamental theorem-
- This approach is rooted in the idea that natural selection, under certain conditions, maximizes the average fitness of individuals in a population.

2.4 Statistical confrontation: finding the maximum

- Explain how to find the maximum of a function with R. Parallel between genetic algorithms and optimization.

Chapter 3

Conflict resolution in ecology and evolution: war and peace

This chapter is under construction. Content may change.

In the previous chapter we looked at the decisions of individual animals assuming that the the decisions where taken by each forager alone. For instance, for the sit-and-wait predator we assumed that the territory could be as large as the predator wanted, and that the optimal territory size was the one where the energy per unit time was maximized. But what happens if the territory is so large that it starts intruding in the territory of the neighbors? Or in other words, what happens if the optimal territory size is for instance 1 m^2 , but the density of the predators is so high that there is more than 1 predator per square meter? Then there is a potential conflict between individuals and it may not be possible to have territory sizes that are optimal. More generally, one can say that the decision of an individual now affects the decisions of its conspecifics.

In evolutionary contexts, where decisions or phenotypes are determined by genes, we enter the so called realm of frequency-dependent selection. In contrasts with frequency independent selection, where the fitness of a genotype depends only on the environment, in frequence-dependent selection the fitness of a genotype depends also on the frequency of the different genotypes in the population.

The analysis of this kind of behavior conflict or evolutionary dynamics is the topic of game theory, particularly non-cooperative game theory. Some of the key concepts of game theory for animal behavior were developed by the biologist John Maynard Smith in the 1970's and beautiful explained in his book "Evolution and the Theory of Games" Maynard Smith (1982). We now know that similar ideas were developed independently by the mathematician John Nash, twenty years earlier in the 1950's, ideas that would lead to Nash being awarded the Nobel Prize of Economics in 1994.

In this chapter we examine three non-cooperative games that model situations of behavior or evolutionary conflict: the prisoner's dillema, the Hawk-Dove game, and the war of attrition. They are called non-cooperative because each "player" does not know what the other "player" is going to do in the game, and the players cannot discuss the strategy that they are going to take in any cooperative game. They are also sometimes described as closed envelope games. This is, each player

should write what "strategy" it adopts in each play without knowing the choice of the other player.

3.1 The prisoner's dillema and the evolution of cooperation

A lot of the empahsis on evolutionary biology and ecology is place on competition, on the idea of "survival of the fittest" and that selection operates at the level of individuals favoring selfish behaviors. However, cooperation is super common in nature, from praire dogs taking turn as sentinels, informing the group of the arrival of a predator, to back scratching in primates. One beautiful explanation for such cooperation is based on the principles of kin selection, developed by Bill Hamilton, or the idea that it makes evolutionary sense to help others if they are genetically related to you, as this increases the probability of the same genes you have to be passed into the next generation. Hamilton's contribution to this problem was seminal because he noted that genetic relatedness can be particularly high between females in haploid-diploid systems such as bees and ants, leading to a widespread of cooperating in rearing offspring and even the abdication of many individuals from reproducing in favor of assisting the queen. But can cooperation also evolve in the absence of genetic relatedness? This is the problem that can be studied with the prisoner's dilemma.

In the prisionner's dilemma game there are two players. it is inspired by a situation where two suspects of being accomplices in a crime are arrested and are being questioned in separate rooms without contact between them. They are faced with the dilemma of cooperating with each other by claimng inocence, or cutting a deal with the investigators by recognizing the responsibility for the crime and by denouncing the other player. If both cooperate they can potetinally get both away, but if one accomplice cooperates and the other defects, then the one that cooperated gets the maximum penalty.

We can formalize the prisoner's dilemma game by stating that each player can choose one of two strategies in each round of the game, defect or cooperate. The pay-off matrix of the Prisionner's Dilemma, from the pespective of player 1 (rows), playing against player 2 (columns) is

Player 1\Player 2	C	D
C	3	0
D	5	1

You can think of the pay-off points as a fitness gain in relation to the fitness before the play. The exact payoffs are not so important, and there are other variants of the game with different pay-offs. Instead, it is the relationship between the different pay-offs that determines the best strategy for the game. If the players knew what the other player would do, the best collective strategy would be to cooperate, as they both would receive 3 points for a total of 6 points. If one of the players defects while the other cooperate they accrue collectively only 5 points, while if both defect they would accrue together 2 points. But they don't know what the other player is going to do (non-cooperative game) and need to make a decision that maximizes their individual fitness. What should they do?

The general solution that both Maynard Smith and Nash came up for these games is that one needs to identify which strategy is the best response to itself. This is called an Evolutionary Stable Strategy (ESS, also known as Nash Equilibrium[1]) because it is the strategy that when dominant

[1]There's a small difference between the two, see (Dugatkin and Reeve 1998).

in the population cannot be invaded by any other strategy. Mathematically speaking, the ESS is the strategy S^* that has the higher payoff when played against itself,

$$\forall S_i, \quad E(S^*, S^*) \geq E(S_i, S^*)$$

3.2 The Hawk-Dove game and animal contests

3.3 The war of atttrition

Chapter 4

Movement and dispersal: should I stay or should I go?

This chapter is under construction. Content may change.

All animals move, at least at a given stage of they life. Animals can move to find food, mates or just a new territory to settle. Some animals even move across continents to track the seasonal changes in resources. A particularly important type of movement is natal dispersal, this is how animals disperse from their birth place to the place where they reproduce. Dispersal also occurs in plants when seeds are transported by wind or by animals. Some movement may be directional, for instance fish following a river or ocean current, while other movements may seem more random.

4.1 The random-walk

One may ask how animals take decisions on where to move and how this influences their distribution over time. One simple but yet effective model is the random-walk model. It assumes that individuals at each moment take a step in a random direction. The question is then, where will the individual be after a certain amount of time? Let's start with a simple random-walk in one dimension. Consider that the initial position of an individual at time $t = 0$ is $x = 0$ and that it can either move a step to the right or a step to the left with probability p, while the probability of staying still is $q = 1 - p$. We keep track of how many individuals with are at each position x at time t with the function n(t,x). It's easy to see that the number of individuals at position x at time t+1 is the number of individuals that stayed still, plus the individuals that came from the left plus the individuals that came from the right, this is,

$$n(x, t + 1) = n(x, t)q + n(x - 1, t)p/2 + n(x + 1, t)p/2 \tag{4.1}$$

Diffusion dispersal models as random-walk.

4.2 Habitat selection

Interactions between neighbor individuals (e.g. competition for space or resources).

4.2.1 Ideal free distribution

4.2.2 Ideal despotic distribution

Part II

Ecology of populations

Chapter 5

Demography: boom or bust

This chapter is under construction. Content may change.

Part III

Ecology of communities

Chapter 6

Measuring biodiversity across scales

This chapter is under construction. Content may change.

A question that has intrigued ecologists for many decades is how to measure biodiversity. Part of the challenge is that the term biodiversity is an holistic term that can be used as a synonym for "life on Earth" and which can be hard to define in quantitative terms. The Convention on Biological Diversity defines it as "the variability among living organisms from all sources including, *inter alia*, terrestrial, marine and other aquatic ecosystems and the ecological complexes of which they are part; this includes diversity within species, between species and of ecosystems" (Article 2) (CBD 1992). This definition states that biodiversity has different levels of organization, from the genetic to the ecosystem level, and emphasizes the variability within each level.



alpha, beta, gamma diversity

Figure on coral reefs

6.1 Species-area relationship

6.2 Species abundance distributions

6.3 Estimating diversity indices

Chapter 7

Modelling biodiversity change: winners and losers

Every minute about 2 football fields of native forest are cleared somewhere in the world. What are the consequences of land-use change for biodiversity?

References

CBD. 1992. "Convention on Biological Diversity."

Dugatkin, Lee Alan, and Hudson Kern Reeve. 1998. *Game Theory & Animal Behavior*. Oxford University Press.

F. Dormann, Carsten, Jana M. McPherson, Miguel B. Araújo, Roger Bivand, Janine Bolliger, Gudrun Carl, Richard G. Davies, et al. 2007. "Methods to Account for Spatial Autocorrelation in the Analysis of Species Distributional Data: A Review." *Ecography* 30 (5): 609–28. https://doi.org/10.1111/j.2007.0906-7590.05171.x.

Fick, Stephen E., and Robert J. Hijmans. 2017. "WorldClim 2: New 1-Km Spatial Resolution Climate Surfaces for Global Land Areas." *International Journal of Climatology* 37 (12): 4302–15. https://doi.org/10.1002/joc.5086.

Guisan, Antoine, Wilfried Thuiller, and Niklaus E. Zimmermann. 2017. *Habitat Suitability and Distribution Models: With Applications in r*. Cambridge University Press.

Hilborn, Ray, and Marc Mangel. 1997. *The Ecological Detective: Confronting Models with Data*. 1st ed. Princeton University Press.

Huey, B. 1982. "Temperature, Physiology, and the Ecology of Reptiles." In, edited by Carl Gans and F. Harvey. New York: Academic Press.

Losos, Jonathan B. 2009. *Lizards in an Evolutionary TreeEcology and Adaptive Radiation of Anoles*. University of California Press. https://doi.org/10.1525/california/9780520255913.001.0001.

Maynard Smith, J. 1982. *Evolution and the Theory of Games*. Cambridge: Cambridge University Press.

Otto, Sarah P., and Troy Day. 2007. *A Biologist's Guide to Mathematical Modeling in Ecology and Evolution*. Princeton University Press. https://doi.org/10.1515/9781400840915.

Pyke, Graham H. 1984. "Optimal Foraging Theory: A Critical Review." *Annual Review of Ecology and Systematics* 15 (January): 523–75. http://www.jstor.org/stable/2096959.

Roughgarden, J. 1998a. *Primer of Ecological Theory*. Upper Saddle River, New Jersey: Prentice Hall.

———. 1998b. *Primer of Ecological Theory*. Upper Saddle River, New Jersey: Prentice Hall.

Shafir, S., and J. Roughgarden. 1998. "Testing Predictions of Foraging Theory for a Sit-and-Wait Forager, {Anolis Gingivinus}." *Behav Ecol* 9 (1Shafir98): 74–84.

Wieczorek, John, Olaf Bánki, Stan Blum, John Deck, Markus Döring, Gabriele Dröge, Dag Endresen, et al. 2014. "Meeting Report: GBIF Hackathon-Workshop on Darwin Core and Sample Data (22-24 May 2013)." *Standards in Genomic Sciences* 9 (3): 585–98. https://doi.org/10.4056/sigs.4898640.

Winchell, Kristin M., Robert Graham Reynolds, Sofia R. Prado-Irwin, Alberto R. Puente-Rolón,

and Liam J. Revell. 2016. "Data from: Phenotypic Shifts in Urban Areas in the Tropical Lizard Anolis Cristatellus." https://doi.org/10.5061/DRYAD.H234N.

Appendix A

A brief R tutorial

This chapter is under construction. Content may change.

If you are new to **R** you can have a short dive into its main features by working through this tutorial. If you had learnt programming in another computer language, you will be able to skim over this tutorial to find the main differences from what you have learnt to how things are done in **R.**

A.1 Basic data structures

A.1.1 Variables

Variables can be any sequence of letter and numbers, but # it cannot start with a number

```
x = 2
x <- 4
2+2
```

```
[1] 4
```

```
y <- x^5
y
```

```
[1] 1024
```

Please note that you can comment code by using the # character.

A.1.2 Vectors

Let's create vectors.

```
# Introduction to vectors
v1 <- c(2,3,6,12)
v2 <- 1:100
length(v2)
```

```
[1] 100
```

```
v2
```

```
 [1]   1   2   3   4   5   6   7   8   9  10  11  12  13  14  15  16  17  18
[19]  19  20  21  22  23  24  25  26  27  28  29  30  31  32  33  34  35  36
[37]  37  38  39  40  41  42  43  44  45  46  47  48  49  50  51  52  53  54
[55]  55  56  57  58  59  60  61  62  63  64  65  66  67  68  69  70  71  72
[73]  73  74  75  76  77  78  79  80  81  82  83  84  85  86  87  88  89  90
[91]  91  92  93  94  95  96  97  98  99 100
```

```
v3 <- seq(1,100,5)  # call without naming arguments
v3
```

```
 [1]  1  6 11 16 21 26 31 36 41 46 51 56 61 66 71 76 81 86 91 96
```

```
v3 <- seq(from=1,to=100,by=5) # call with names of arguments
v3
```

```
 [1]  1  6 11 16 21 26 31 36 41 46 51 56 61 66 71 76 81 86 91 96
```

```
v3 <- seq(to=100,by=5) # call skipping the first argument
#and using the default value 1 - see help(seq)
v3
```

```
 [1]  1  6 11 16 21 26 31 36 41 46 51 56 61 66 71 76 81 86 91 96
```

```
v3 <- seq(by=5,to=100) # call by arguments and change order or arguments
```

How to index vectors?

```
# Indexing vectors
v3[3] #uses square brackets to obtain the third element of the vector
```

```
[1] 11
```

```
v3>20 # produce a vector of boolean values that are TRUE when
```

```
 [1] FALSE FALSE FALSE FALSE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE
[13]  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE
```

```
      #v3 is greater than 20
v3[v3>20] # select from v3 all the values that are greater than 20
```

```
 [1] 21 26 31 36 41 46 51 56 61 66 71 76 81 86 91 96
```

```
v4<-c(1,2,3,4,5)
v4[c(FALSE,TRUE,FALSE,TRUE,FALSE)] #select from v4 the second and fourth element
```

```
[1] 2 4
```

```
v3[1:10] # first ten elements
```

```
 [1]  1  6 11 16 21 26 31 36 41 46
```

```
v3[-1] # dropping first element
```

```
 [1]  6 11 16 21 26 31 36 41 46 51 56 61 66 71 76 81 86 91 96
```

```
head(v2) # prints the first few elements of v2
```

```
[1] 1 2 3 4 5 6
```

```
tail(v2) # prints the last few elements of v2
```

```
[1]  95  96  97  98  99 100
```

```
which(v3 == 26) # returns the position of v3 that equals 26
```

```
[1] 6
```

What kind of numerical operations are possible on vectors?

```
2^v2
```

```
  [1] 2.000000e+00 4.000000e+00 8.000000e+00 1.600000e+01 3.200000e+01
  [6] 6.400000e+01 1.280000e+02 2.560000e+02 5.120000e+02 1.024000e+03
 [11] 2.048000e+03 4.096000e+03 8.192000e+03 1.638400e+04 3.276800e+04
 [16] 6.553600e+04 1.310720e+05 2.621440e+05 5.242880e+05 1.048576e+06
 [21] 2.097152e+06 4.194304e+06 8.388608e+06 1.677722e+07 3.355443e+07
 [26] 6.710886e+07 1.342177e+08 2.684355e+08 5.368709e+08 1.073742e+09
 [31] 2.147484e+09 4.294967e+09 8.589935e+09 1.717987e+10 3.435974e+10
 [36] 6.871948e+10 1.374390e+11 2.748779e+11 5.497558e+11 1.099512e+12
 [41] 2.199023e+12 4.398047e+12 8.796093e+12 1.759219e+13 3.518437e+13
 [46] 7.036874e+13 1.407375e+14 2.814750e+14 5.629500e+14 1.125900e+15
 [51] 2.251800e+15 4.503600e+15 9.007199e+15 1.801440e+16 3.602880e+16
 [56] 7.205759e+16 1.441152e+17 2.882304e+17 5.764608e+17 1.152922e+18
 [61] 2.305843e+18 4.611686e+18 9.223372e+18 1.844674e+19 3.689349e+19
 [66] 7.378698e+19 1.475740e+20 2.951479e+20 5.902958e+20 1.180592e+21
 [71] 2.361183e+21 4.722366e+21 9.444733e+21 1.888947e+22 3.777893e+22
 [76] 7.555786e+22 1.511157e+23 3.022315e+23 6.044629e+23 1.208926e+24
 [81] 2.417852e+24 4.835703e+24 9.671407e+24 1.934281e+25 3.868563e+25
 [86] 7.737125e+25 1.547425e+26 3.094850e+26 6.189700e+26 1.237940e+27
 [91] 2.475880e+27 4.951760e+27 9.903520e+27 1.980704e+28 3.961408e+28
 [96] 7.922816e+28 1.584563e+29 3.169127e+29 6.338253e+29 1.267651e+30
```

```
log(v2)
```

```
  [1] 0.0000000 0.6931472 1.0986123 1.3862944 1.6094379 1.7917595 1.9459101
  [8] 2.0794415 2.1972246 2.3025851 2.3978953 2.4849066 2.5649494 2.6390573
 [15] 2.7080502 2.7725887 2.8332133 2.8903718 2.9444390 2.9957323 3.0445224
 [22] 3.0910425 3.1354942 3.1780538 3.2188758 3.2580965 3.2958369 3.3322045
 [29] 3.3672958 3.4011974 3.4339872 3.4657359 3.4965076 3.5263605 3.5553481
 [36] 3.5835189 3.6109179 3.6375862 3.6635616 3.6888795 3.7135721 3.7376696
 [43] 3.7612001 3.7841896 3.8066625 3.8286414 3.8501476 3.8712010 3.8918203
```

```
 [50] 3.9120230 3.9318256 3.9512437 3.9702919 3.9889840 4.0073332 4.0253517
 [57] 4.0430513 4.0604430 4.0775374 4.0943446 4.1108739 4.1271344 4.1431347
 [64] 4.1588831 4.1743873 4.1896547 4.2046926 4.2195077 4.2341065 4.2484952
 [71] 4.2626799 4.2766661 4.2904594 4.3040651 4.3174881 4.3307333 4.3438054
 [78] 4.3567088 4.3694479 4.3820266 4.3944492 4.4067192 4.4188406 4.4308168
 [85] 4.4426513 4.4543473 4.4659081 4.4773368 4.4886364 4.4998097 4.5108595
 [92] 4.5217886 4.5325995 4.5432948 4.5538769 4.5643482 4.5747110 4.5849675
 [99] 4.5951199 4.6051702
```

```
v5 <- 101:200
v5/v2
```

```
  [1] 101.000000  51.000000  34.333333  26.000000  21.000000  17.666667
  [7]  15.285714  13.500000  12.111111  11.000000  10.090909   9.333333
 [13]   8.692308   8.142857   7.666667   7.250000   6.882353   6.555556
 [19]   6.263158   6.000000   5.761905   5.545455   5.347826   5.166667
 [25]   5.000000   4.846154   4.703704   4.571429   4.448276   4.333333
 [31]   4.225806   4.125000   4.030303   3.941176   3.857143   3.777778
 [37]   3.702703   3.631579   3.564103   3.500000   3.439024   3.380952
 [43]   3.325581   3.272727   3.222222   3.173913   3.127660   3.083333
 [49]   3.040816   3.000000   2.960784   2.923077   2.886792   2.851852
 [55]   2.818182   2.785714   2.754386   2.724138   2.694915   2.666667
 [61]   2.639344   2.612903   2.587302   2.562500   2.538462   2.515152
 [67]   2.492537   2.470588   2.449275   2.428571   2.408451   2.388889
 [73]   2.369863   2.351351   2.333333   2.315789   2.298701   2.282051
 [79]   2.265823   2.250000   2.234568   2.219512   2.204819   2.190476
 [85]   2.176471   2.162791   2.149425   2.136364   2.123596   2.111111
 [91]   2.098901   2.086957   2.075269   2.063830   2.052632   2.041667
 [97]   2.030928   2.020408   2.010101   2.000000
```

A.1.3 Strings

```
#Using strings in R
mystring <- "Ecology"
vstrg <- c("Anna", "Peter", "Xavier")
vstrg[2]
```

```
[1] "Peter"
```

A.1.4 Matrices

```
m <- matrix(5,3,2)
m
```

```
     [,1] [,2]
[1,]    5    5
[2,]    5    5
```

```
[3,]    5    5
```

```
m2 <- matrix(1:6,3,2)
m2
```

```
     [,1] [,2]
[1,]    1    4
[2,]    2    5
[3,]    3    6
```

```
t(m2) # transposes matrix
```

```
     [,1] [,2] [,3]
[1,]    1    2    3
[2,]    4    5    6
```

```
x <- 1:4
y <- 5:8

m3<-cbind(x,y)
m3
```

```
     x y
[1,] 1 5
[2,] 2 6
[3,] 3 7
[4,] 4 8
```

```
m4<-rbind(x,y)
m4
```

```
  [,1] [,2] [,3] [,4]
x    1    2    3    4
y    5    6    7    8
```

```
# Indexing matrices
m3[3,2] #element in row 3 and column 2
```

```
y
7
```

```
m3[1,] #entire first row
```

```
x y
1 5
```

```
m3[,1] #entire first column
```

```
[1] 1 2 3 4
```

```
colnames(m3)<-c("col1","col2")
m3
```

```
     col1 col2
[1,]    1    5
[2,]    2    6
[3,]    3    7
[4,]    4    8
```

```
m3[,"col2"]
```

```
[1] 5 6 7 8
```

A.1.5 Lists

```
# Lists in R
mylist <- list(elem1=m,elem2=v2,elem3="my list")
mylist$elem2
```

```
  [1]   1   2   3   4   5   6   7   8   9  10  11  12  13  14  15  16  17  18
 [19]  19  20  21  22  23  24  25  26  27  28  29  30  31  32  33  34  35  36
 [37]  37  38  39  40  41  42  43  44  45  46  47  48  49  50  51  52  53  54
 [55]  55  56  57  58  59  60  61  62  63  64  65  66  67  68  69  70  71  72
 [73]  73  74  75  76  77  78  79  80  81  82  83  84  85  86  87  88  89  90
 [91]  91  92  93  94  95  96  97  98  99 100
```

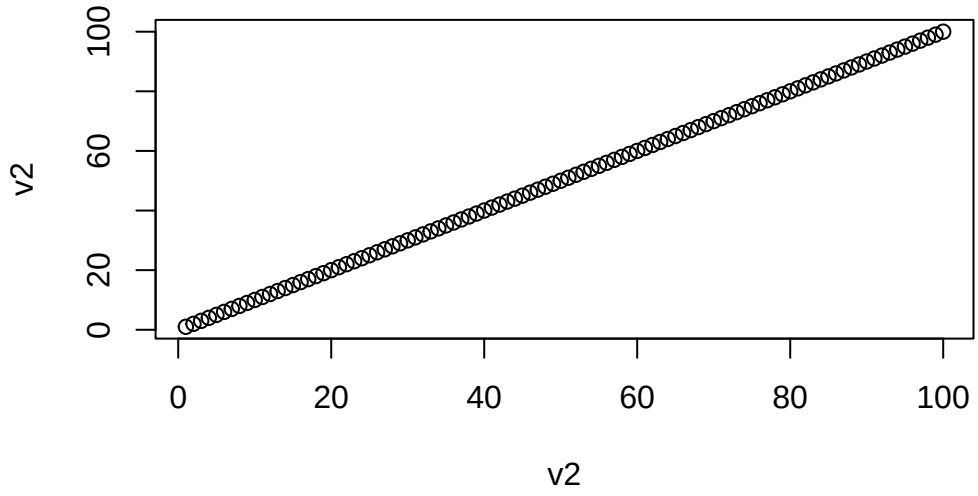
A.1.6 Dataframes

```
# Dataframes
df <- as.data.frame(m3)
df$col1
```

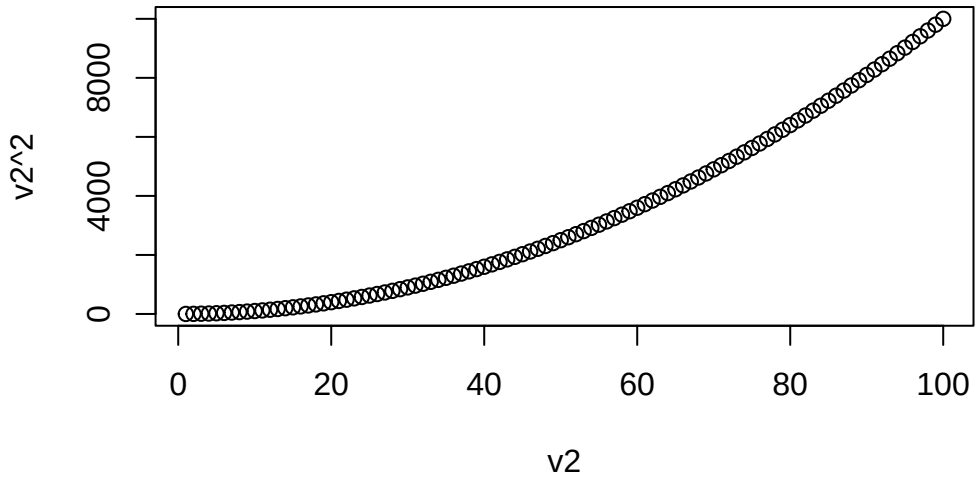
```
[1] 1 2 3 4
```

A.2 Basic plots in R

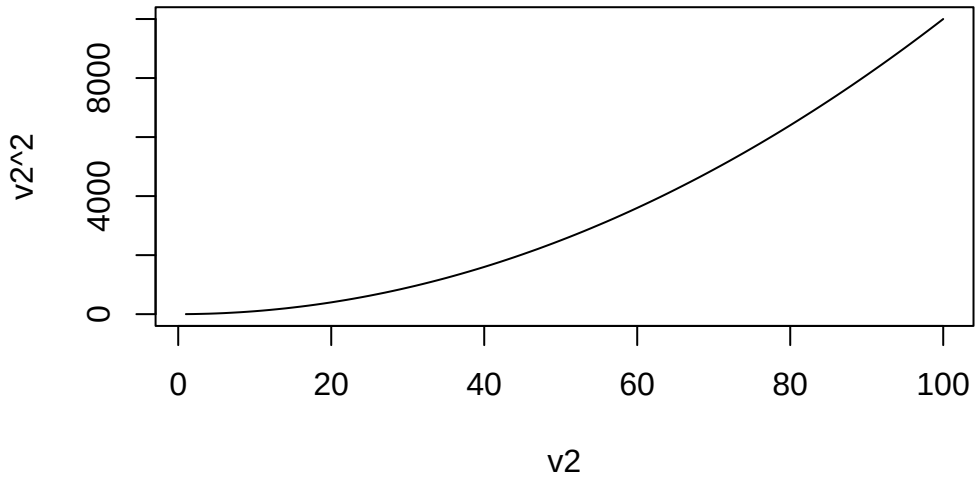
```
#making plots in R
plot(v2,v2)
```



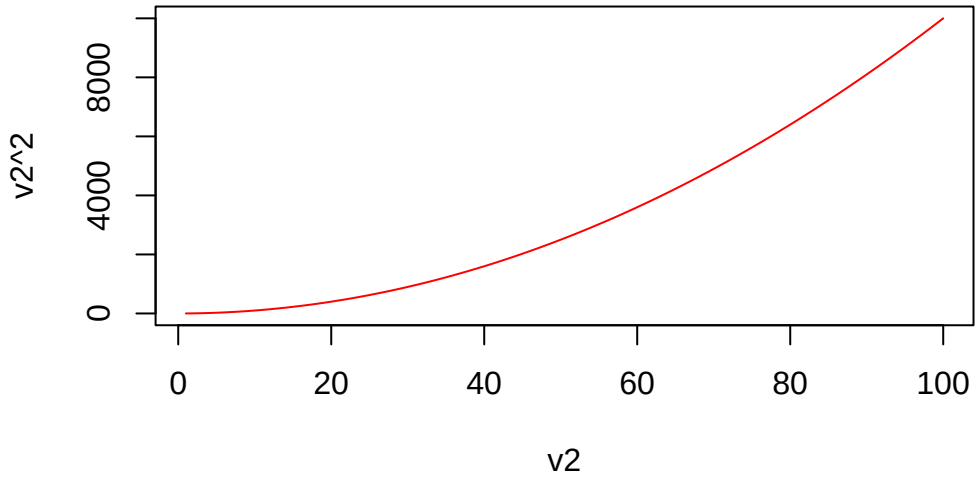

```
plot(v2,v2^2)
```



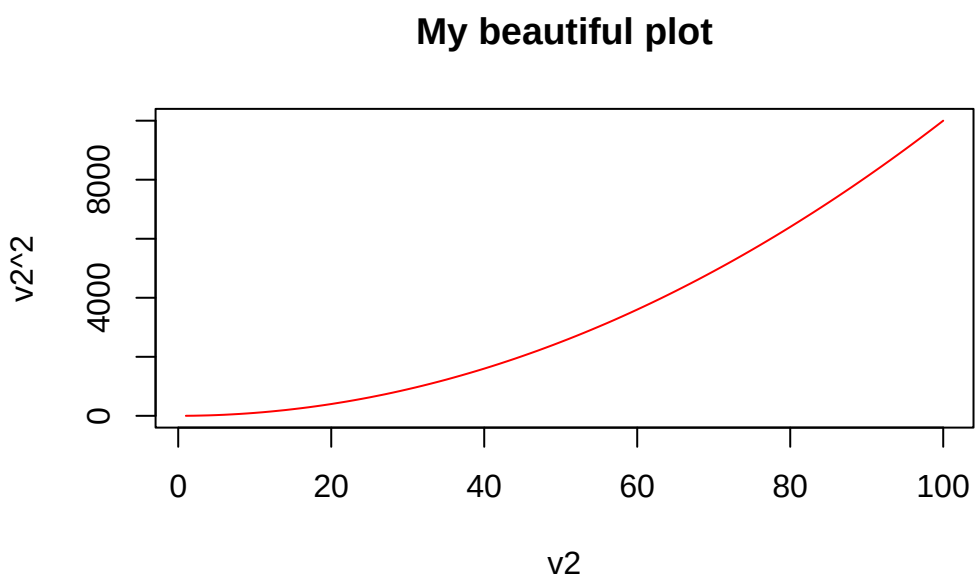

```
plot(v2,v2^2,type="l")
```



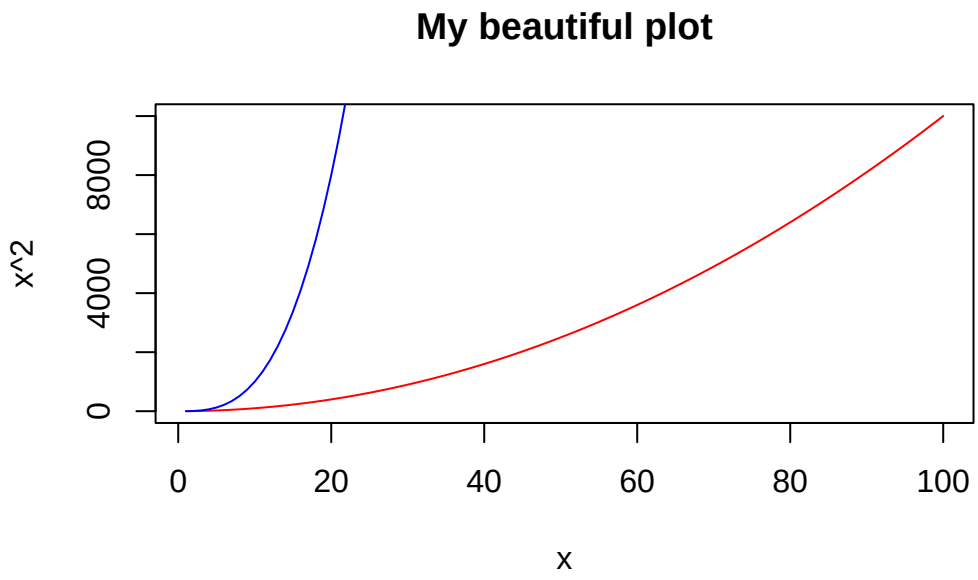

```
plot(v2,v2^2,type="l",col="red")
```




```
plot(v2,v2^2,type="l",col="red",main="My beautiful plot")
```




```
plot(v2,v2^2,type="l",col="red",main="My beautiful plot",xlab="x",
     ylab="x^2")
lines(v2,v2^3,col="blue")
```



A.3 Iterations and conditional expressions

```
# FOR loops
```

```
for (k in 1:10)  # for k =1, 2, 3, 4, 5,...10
  print (k^2)   #do this
```

```
[1] 1
[1] 4
[1] 9
[1] 16
[1] 25
[1] 36
[1] 49
[1] 64
[1] 81
[1] 100
```

```
R <- 1.2
n <- 1
print(n[1])
```

```
[1] 1
```

```
for (t in 1:100)
{
  n[t+1] <- R*n[t]
  print(n[t+1])
}
```

```
[1] 1.2
[1] 1.44
[1] 1.728
[1] 2.0736
[1] 2.48832
[1] 2.985984
[1] 3.583181
[1] 4.299817
[1] 5.15978
[1] 6.191736
[1] 7.430084
[1] 8.9161
[1] 10.69932
[1] 12.83918
[1] 15.40702
[1] 18.48843
[1] 22.18611
[1] 26.62333
[1] 31.948
[1] 38.3376
[1] 46.00512
```

```
[1] 55.20614
[1] 66.24737
[1] 79.49685
[1] 95.39622
[1] 114.4755
[1] 137.3706
[1] 164.8447
[1] 197.8136
[1] 237.3763
[1] 284.8516
[1] 341.8219
[1] 410.1863
[1] 492.2235
[1] 590.6682
[1] 708.8019
[1] 850.5622
[1] 1020.675
[1] 1224.81
[1] 1469.772
[1] 1763.726
[1] 2116.471
[1] 2539.765
[1] 3047.718
[1] 3657.262
[1] 4388.714
[1] 5266.457
[1] 6319.749
[1] 7583.698
[1] 9100.438
[1] 10920.53
[1] 13104.63
[1] 15725.56
[1] 18870.67
[1] 22644.8
[1] 27173.76
[1] 32608.52
[1] 39130.22
[1] 46956.26
[1] 56347.51
[1] 67617.02
[1] 81140.42
[1] 97368.5
[1] 116842.2
[1] 140210.6
[1] 168252.8
[1] 201903.3
```

```
[1] 242284
[1] 290740.8
[1] 348889
[1] 418666.7
[1] 502400.1
[1] 602880.1
[1] 723456.1
[1] 868147.4
[1] 1041777
[1] 1250132
[1] 1500159
[1] 1800190
[1] 2160228
[1] 2592274
[1] 3110729
[1] 3732875
[1] 4479450
[1] 5375340
[1] 6450408
[1] 7740489
[1] 9288587
[1] 11146304
[1] 13375565
[1] 16050678
[1] 19260814
[1] 23112977
[1] 27735572
[1] 33282687
[1] 39939224
[1] 47927069
[1] 57512482
[1] 69014979
[1] 82817975
```

```
R <- 1.2
n <- 1
for (t in 1:100)
  n[t+1] <- R*n[t]

# IF conditional statement

# logical operators
# == equal to
# > greater than
# < smaller than
# >= greater or equal
# <= smaller or equal
```

```
# != different from
# && and
# || or

if (3>2) print ("yes")
```

```
[1] "yes"
```

```
if (3==2) print ("yes") else print("no")
```

```
[1] "no"
```

```
if ((3>2)&&(4>5)) print ("yes")

for (k in 1:10)  # for k =1, 2, 3, 4, 5,...10
  if (k^2>20) print (k^2)
```

```
[1] 25
[1] 36
[1] 49
[1] 64
[1] 81
[1] 100
```

A.4 Writing functions

```
# creating FUNCTIONS in r

pythagoras <- function (c1,c2)
{
  h <- sqrt (c1^2 + c2^2)
  return (h)
}

pythagoras(1,1)
```

```
[1] 1.414214
```

```
pythagoras(5,5)
```

```
[1] 7.071068
```

```
pythagoras(10,1)
```

```
[1] 10.04988
```

```
# regression in R

help(lm)
```

```
x <- c(1,2,3,4)
y <- c(1.1,2.3,2.9,4.1)
plot(x,y)
myreg<-lm(y ~ x)
summary(myreg)
```

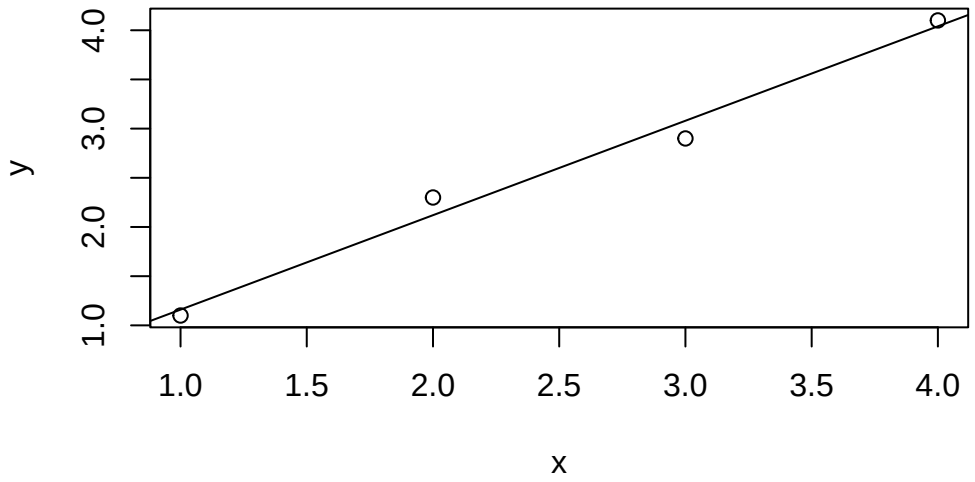
```
Call:
lm(formula = y ~ x)

Residuals:
    1     2     3     4
-0.06  0.18 -0.18  0.06

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept)  0.20000    0.23238   0.861  0.48012
x            0.96000    0.08485  11.314  0.00772 **
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 0.1897 on 2 degrees of freedom
Multiple R-squared:  0.9846,    Adjusted R-squared:  0.9769
F-statistic:   128 on 1 and 2 DF,  p-value: 0.007722
```

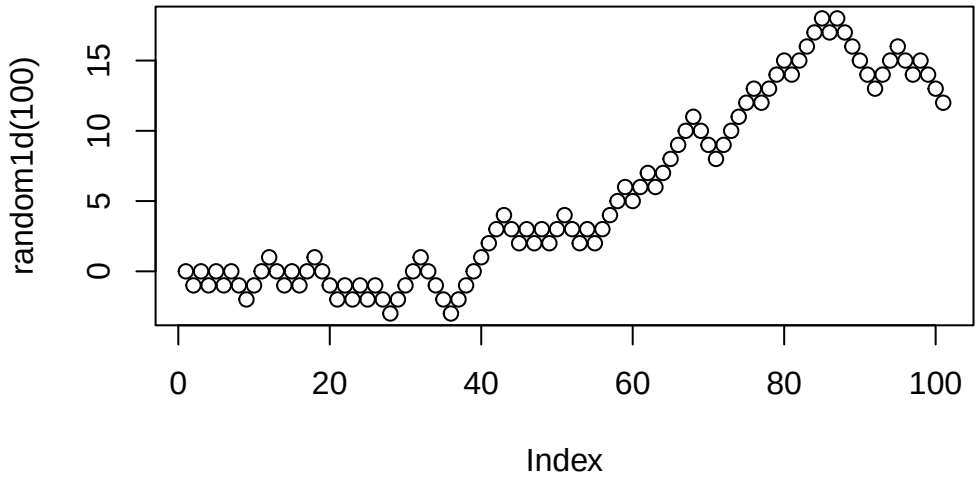
```
abline(myreg)
```



A.5 Random numbers and statistical distributions

```
random1d<-function(tmax)
{
x<-0
for (t in 1:tmax)
{
  r<-runif(1)
  if (r<1/2)
    x[t+1]<-x[t]+1 else
      x[t+1]<-x[t]-1
}
return(x)
}

plot(random1d(100))
```




```
tmax<-10000
lastx<-0
for (i in 1:1000)
{
  x<-random1d(tmax)
  lastx[i]<-x[tmax]
}

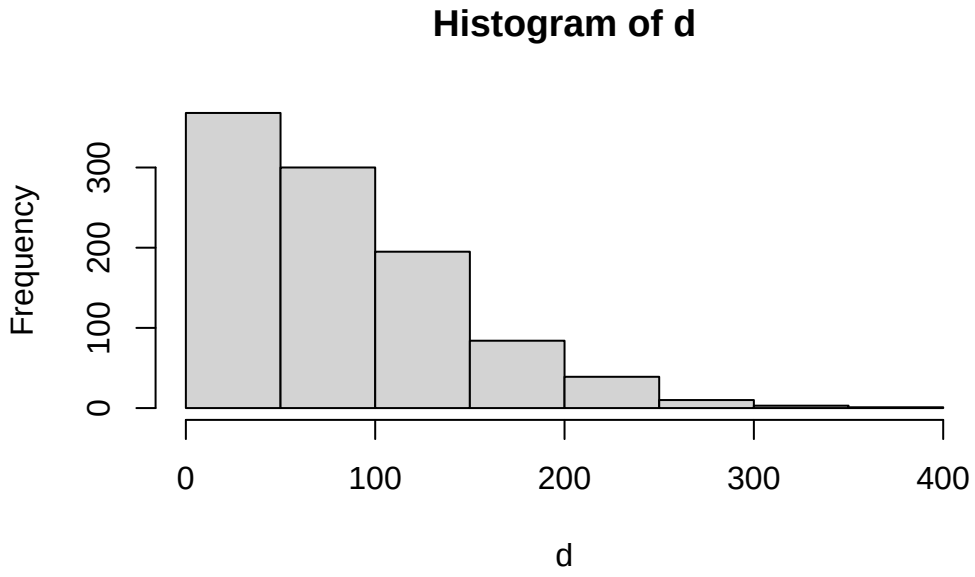
hist(lastx)
```




```
mean(lastx)
```

```
[1] -2.648
```

```
d<-sqrt(lastx^2)
hist(d)
```




```
mean(d)
```

```
[1] 81.818
```

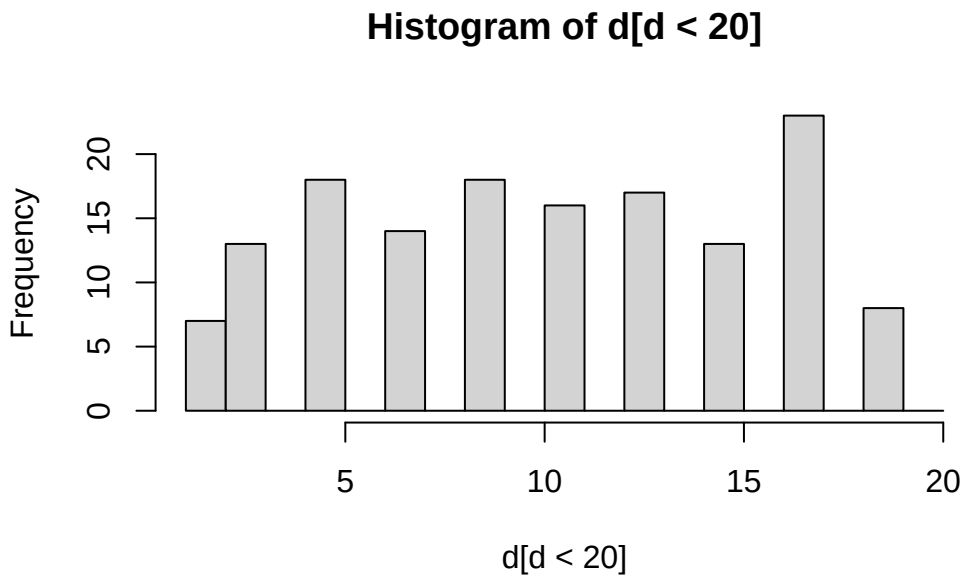
```
median(d)
```

```
[1] 71
```

```
max(d)
```

```
[1] 361
```

```
hist(d[d<20],breaks=c(1:20))
```



A.6 Intro to apply, pipes, ggplot2, tidyverse

Use of apply instead of for loops.

```
# a recursive function that calculates a factorial
myfun <- function(x)
{
  if (x==1)
    return (1)
  else return(x*myfun(x-1))
}
```

The following does not work

```
myfun(1:10) # does not work
```

One needs to do a for loop or use apply

```
#option1 - with a for loop
start_time <- Sys.time()
```

```
y<-0
for (i in 1:100)
  y[i]<-myfun(i)
end_time <- Sys.time()
end_time-start_time
```

```
Time difference of 0.0206449 secs
```

```
y
```

```
  [1]  1.000000e+00  2.000000e+00  6.000000e+00  2.400000e+01  1.200000e+02
  [6]  7.200000e+02  5.040000e+03  4.032000e+04  3.628800e+05  3.628800e+06
 [11]  3.991680e+07  4.790016e+08  6.227021e+09  8.717829e+10  1.307674e+12
 [16]  2.092279e+13  3.556874e+14  6.402374e+15  1.216451e+17  2.432902e+18
 [21]  5.109094e+19  1.124001e+21  2.585202e+22  6.204484e+23  1.551121e+25
 [26]  4.032915e+26  1.088887e+28  3.048883e+29  8.841762e+30  2.652529e+32
 [31]  8.222839e+33  2.631308e+35  8.683318e+36  2.952328e+38  1.033315e+40
 [36]  3.719933e+41  1.376375e+43  5.230226e+44  2.039788e+46  8.159153e+47
 [41]  3.345253e+49  1.405006e+51  6.041526e+52  2.658272e+54  1.196222e+56
 [46]  5.502622e+57  2.586232e+59  1.241392e+61  6.082819e+62  3.041409e+64
 [51]  1.551119e+66  8.065818e+67  4.274883e+69  2.308437e+71  1.269640e+73
 [56]  7.109986e+74  4.052692e+76  2.350561e+78  1.386831e+80  8.320987e+81
 [61]  5.075802e+83  3.146997e+85  1.982608e+87  1.268869e+89  8.247651e+90
 [66]  5.443449e+92  3.647111e+94  2.480036e+96  1.711225e+98 1.197857e+100
 [71] 8.504786e+101 6.123446e+103 4.470115e+105 3.307885e+107 2.480914e+109
 [76] 1.885495e+111 1.451831e+113 1.132428e+115 8.946182e+116 7.156946e+118
 [81] 5.797126e+120 4.753643e+122 3.945524e+124 3.314240e+126 2.817104e+128
 [86] 2.422710e+130 2.107757e+132 1.854826e+134 1.650796e+136 1.485716e+138
 [91] 1.352002e+140 1.243841e+142 1.156773e+144 1.087366e+146 1.032998e+148
 [96] 9.916779e+149 9.619276e+151 9.426890e+153 9.332622e+155 9.332622e+157
```

```
#option 2 - with apply
start_time <- Sys.time()
y<-sapply(1:100,myfun)
end_time <- Sys.time()
end_time-start_time
```

```
Time difference of 0.007334948 secs
```

```
y
```

```
  [1]  1.000000e+00  2.000000e+00  6.000000e+00  2.400000e+01  1.200000e+02
  [6]  7.200000e+02  5.040000e+03  4.032000e+04  3.628800e+05  3.628800e+06
 [11]  3.991680e+07  4.790016e+08  6.227021e+09  8.717829e+10  1.307674e+12
 [16]  2.092279e+13  3.556874e+14  6.402374e+15  1.216451e+17  2.432902e+18
 [21]  5.109094e+19  1.124001e+21  2.585202e+22  6.204484e+23  1.551121e+25
 [26]  4.032915e+26  1.088887e+28  3.048883e+29  8.841762e+30  2.652529e+32
 [31]  8.222839e+33  2.631308e+35  8.683318e+36  2.952328e+38  1.033315e+40
 [36]  3.719933e+41  1.376375e+43  5.230226e+44  2.039788e+46  8.159153e+47
```

```
 [41]  3.345253e+49  1.405006e+51  6.041526e+52  2.658272e+54  1.196222e+56
 [46]  5.502622e+57  2.586232e+59  1.241392e+61  6.082819e+62  3.041409e+64
 [51]  1.551119e+66  8.065818e+67  4.274883e+69  2.308437e+71  1.269640e+73
 [56]  7.109986e+74  4.052692e+76  2.350561e+78  1.386831e+80  8.320987e+81
 [61]  5.075802e+83  3.146997e+85  1.982608e+87  1.268869e+89  8.247651e+90
 [66]  5.443449e+92  3.647111e+94  2.480036e+96  1.711225e+98 1.197857e+100
 [71] 8.504786e+101 6.123446e+103 4.470115e+105 3.307885e+107 2.480914e+109
 [76] 1.885495e+111 1.451831e+113 1.132428e+115 8.946182e+116 7.156946e+118
 [81] 5.797126e+120 4.753643e+122 3.945524e+124 3.314240e+126 2.817104e+128
 [86] 2.422710e+130 2.107757e+132 1.854826e+134 1.650796e+136 1.485716e+138
 [91] 1.352002e+140 1.243841e+142 1.156773e+144 1.087366e+146 1.032998e+148
 [96] 9.916779e+149 9.619276e+151 9.426890e+153 9.332622e+155 9.332622e+157
```

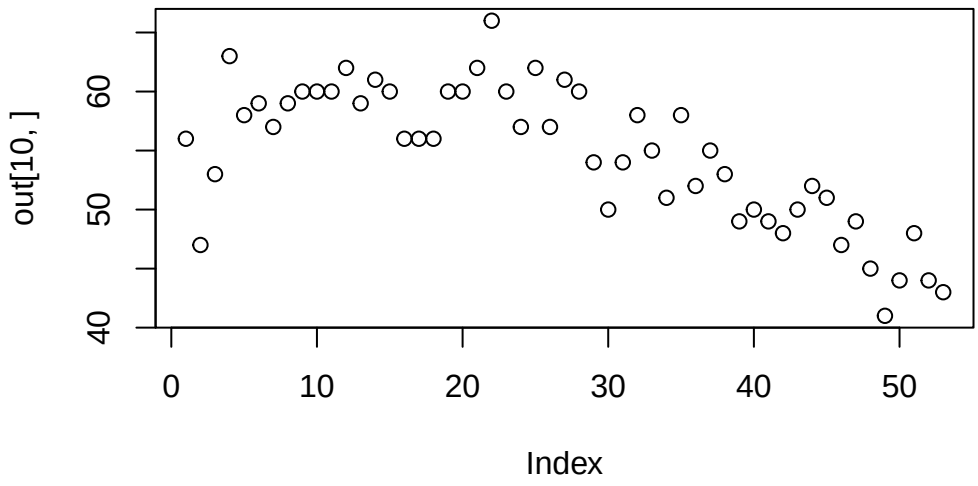
Selecting a subset from a matrix and applying a function to a column of that subset

```
Florida <- read.csv("Labs/Florida.csv")

# number of species for year 1970 and route 20
x=tapply(Florida$Abundance,Florida$Route==20 & Florida$Year==1970, length)
#applies length() to Florida$Abundance but by two levels, when the formula is true
#and when the formula is false

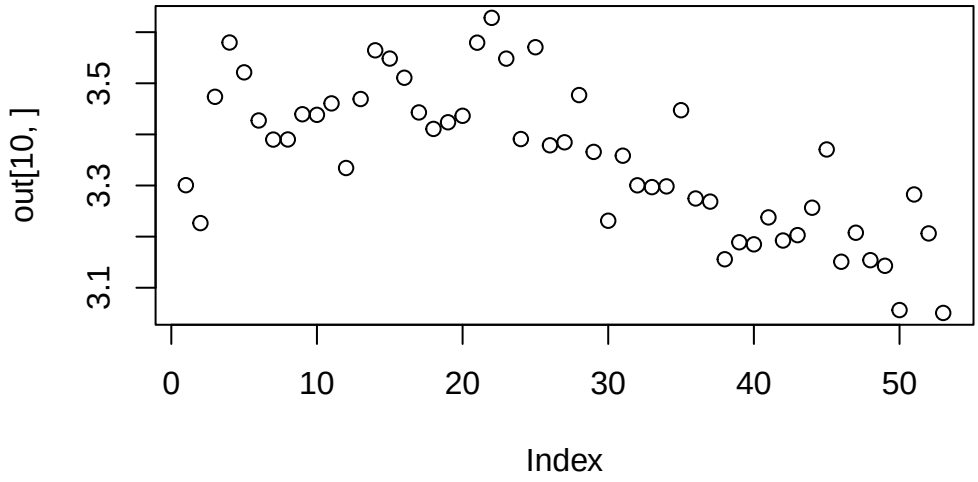
# matrix with number of species per route and per year
out<-tapply(Florida$Abundance,list(Florida$Route,Florida$Year), length)

plot(out[10,])
```




```
shannon<-function(x)
{
  p<-x/sum(x)
  - sum(p*log(p))
}
```

```
out<-tapply(Florida$Abundance,list(Florida$Route,Florida$Year), shannon)
plot(out[10,])
```

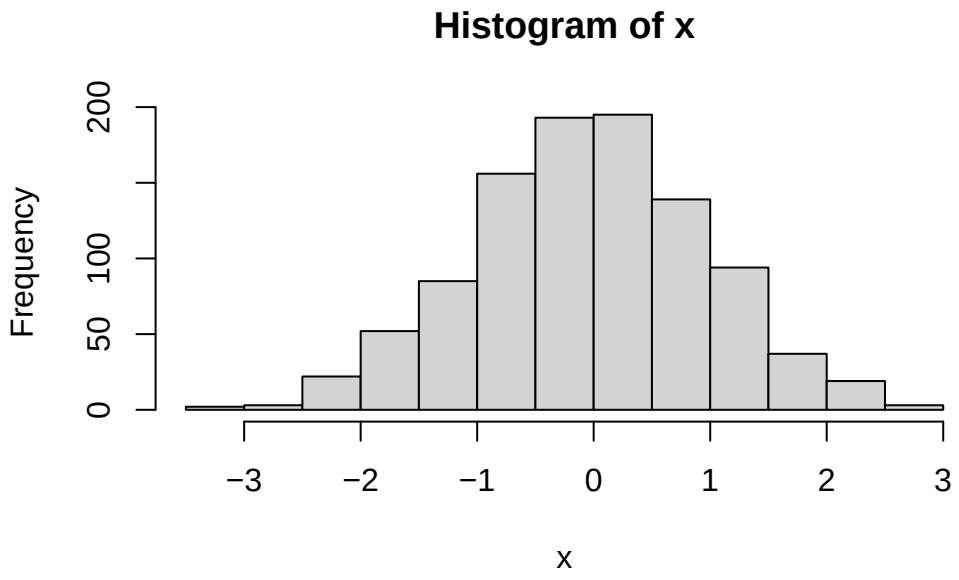


A.6.1 Working with pipes

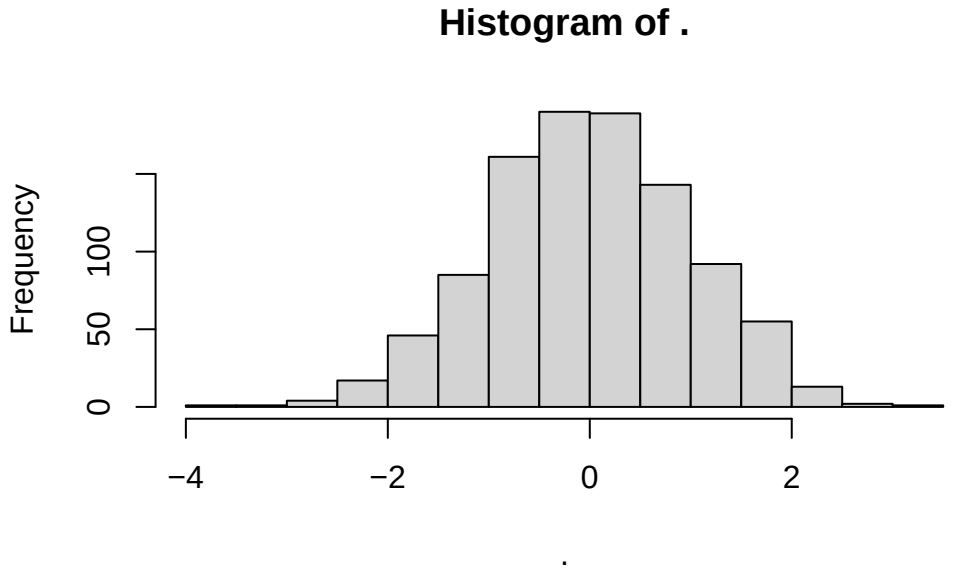
```
library(tidyverse)
```

```
-- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
v dplyr     1.2.0     v readr     2.1.5
v forcats   1.0.1     v stringr   1.5.2
v ggplot2   4.0.0     v tibble    3.3.0
v lubridate 1.9.4     v tidyr     1.3.1
v purrr     1.2.1
-- Conflicts ------------------------------------------ tidyverse_conflicts() --
x dplyr::filter() masks stats::filter()
x dplyr::lag()    masks stats::lag()
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become error
```

```
#our first pipe
x<-rnorm(1000)
hist(x)
```




```
rnorm(1000) %>%  hist
```



Now with the Florida data

```
t<-1:ncol(out)

plot(out[10,])
myreg<-lm(out[10,]~t)
summary(myreg)
```

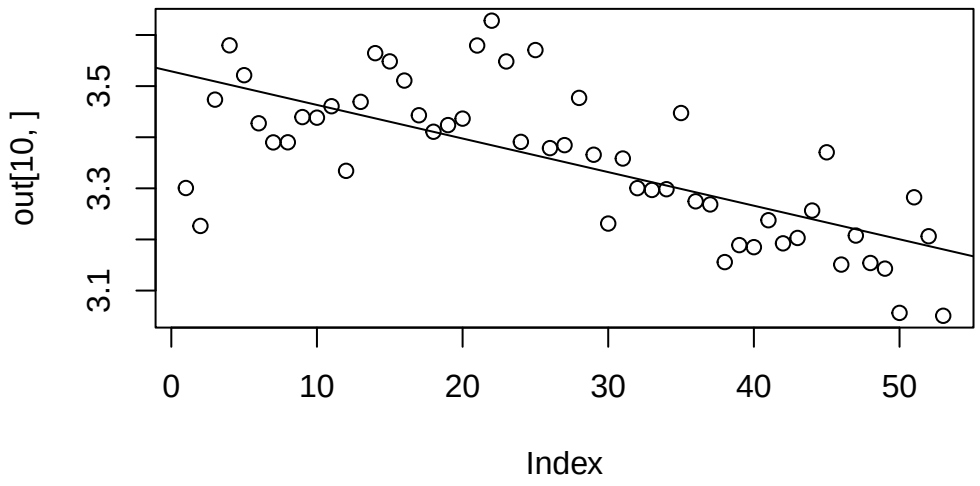
```
Call:
lm(formula = out[10, ] ~ t)

Residuals:
      Min        1Q    Median        3Q       Max
-0.289301 -0.062193 -0.007159  0.033154  0.243719

Coefficients:
             Estimate Std. Error t value Pr(>|t|)
(Intercept)  3.5288806  0.0289650  121.83  < 2e-16 ***
t           -0.0065709  0.0009334   -7.04 4.71e-09 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

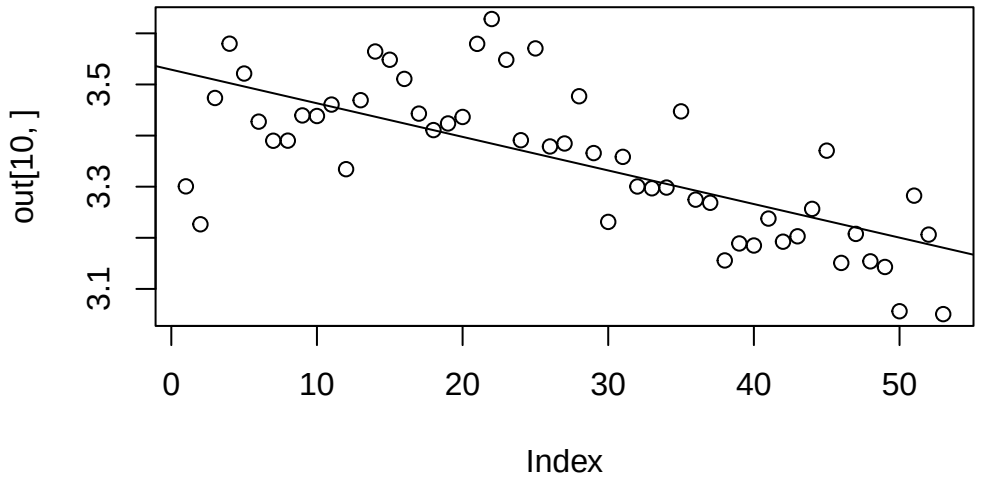
Residual standard error: 0.1039 on 51 degrees of freedom
Multiple R-squared:  0.4928,    Adjusted R-squared:  0.4829
F-statistic: 49.56 on 1 and 51 DF,  p-value: 4.708e-09
```

```
abline(myreg)
```




```
plot(out[10,])
lm(out[10,]~t) %>% summary %>%abline
```

```
Warning in abline(.): only using the first two of 8 regression coefficients
```



A.6.2 Using dplyr (tidyverse)

Base R

```
mtcars
```

	mpg	cyl	disp	hp	drat	wt	qsec	vs	am	gear	carb
Mazda RX4	21.0	6	160.0	110	3.90	2.620	16.46	0	1	4	4
Mazda RX4 Wag	21.0	6	160.0	110	3.90	2.875	17.02	0	1	4	4
Datsun 710	22.8	4	108.0	93	3.85	2.320	18.61	1	1	4	1
Hornet 4 Drive	21.4	6	258.0	110	3.08	3.215	19.44	1	0	3	1
Hornet Sportabout	18.7	8	360.0	175	3.15	3.440	17.02	0	0	3	2
Valiant	18.1	6	225.0	105	2.76	3.460	20.22	1	0	3	1
Duster 360	14.3	8	360.0	245	3.21	3.570	15.84	0	0	3	4
Merc 240D	24.4	4	146.7	62	3.69	3.190	20.00	1	0	4	2
Merc 230	22.8	4	140.8	95	3.92	3.150	22.90	1	0	4	2
Merc 280	19.2	6	167.6	123	3.92	3.440	18.30	1	0	4	4
Merc 280C	17.8	6	167.6	123	3.92	3.440	18.90	1	0	4	4
Merc 450SE	16.4	8	275.8	180	3.07	4.070	17.40	0	0	3	3
Merc 450SL	17.3	8	275.8	180	3.07	3.730	17.60	0	0	3	3
Merc 450SLC	15.2	8	275.8	180	3.07	3.780	18.00	0	0	3	3
Cadillac Fleetwood	10.4	8	472.0	205	2.93	5.250	17.98	0	0	3	4
Lincoln Continental	10.4	8	460.0	215	3.00	5.424	17.82	0	0	3	4
Chrysler Imperial	14.7	8	440.0	230	3.23	5.345	17.42	0	0	3	4
Fiat 128	32.4	4	78.7	66	4.08	2.200	19.47	1	1	4	1
Honda Civic	30.4	4	75.7	52	4.93	1.615	18.52	1	1	4	2
Toyota Corolla	33.9	4	71.1	65	4.22	1.835	19.90	1	1	4	1
Toyota Corona	21.5	4	120.1	97	3.70	2.465	20.01	1	0	3	1
Dodge Challenger	15.5	8	318.0	150	2.76	3.520	16.87	0	0	3	2
AMC Javelin	15.2	8	304.0	150	3.15	3.435	17.30	0	0	3	2
Camaro Z28	13.3	8	350.0	245	3.73	3.840	15.41	0	0	3	4

```
Pontiac Firebird    19.2   8 400.0 175 3.08 3.845 17.05  0  0    3    2
Fiat X1-9           27.3   4  79.0  66 4.08 1.935 18.90  1  1    4    1
Porsche 914-2       26.0   4 120.3  91 4.43 2.140 16.70  0  1    5    2
Lotus Europa        30.4   4  95.1 113 3.77 1.513 16.90  1  1    5    2
Ford Pantera L      15.8   8 351.0 264 4.22 3.170 14.50  0  1    5    4
Ferrari Dino        19.7   6 145.0 175 3.62 2.770 15.50  0  1    5    6
Maserati Bora       15.0   8 301.0 335 3.54 3.570 14.60  0  1    5    8
Volvo 142E          21.4   4 121.0 109 4.11 2.780 18.60  1  1    4    2
```

```r
# 1. Filter rows where mpg > 20
subset_data <- mtcars[mtcars$mpg > 20, ]

# 2. Select only mpg, cyl, hp columns
subset_data <- subset_data[, c("mpg", "cyl", "hp")]

# 3. Add new column kpg = mpg * 1.6
subset_data$kpg <- subset_data$mpg * 1.6

# 4. Order by descending mpg
subset_data <- subset_data[order(-subset_data$mpg), ]
```

Or

```r
subset_data <- subset(mtcars, mpg > 20, select = c(mpg, cyl, hp))
subset_data <- transform(subset_data, kpg = mpg * 1.6)
subset_data <- subset_data[order(-subset_data$mpg), ]
```

Now with tidyverse

```r
library(dplyr)

mtcars |>
    filter(mpg > 20) |>
    dplyr::select(mpg, cyl, hp) |>
    mutate(kpg = mpg * 1.6) |>arrange(desc(mpg))
```

```
                mpg cyl  hp   kpg
Toyota Corolla 33.9   4  65 54.24
Fiat 128       32.4   4  66 51.84
Honda Civic    30.4   4  52 48.64
Lotus Europa   30.4   4 113 48.64
Fiat X1-9      27.3   4  66 43.68
Porsche 914-2  26.0   4  91 41.60
Merc 240D      24.4   4  62 39.04
Datsun 710     22.8   4  93 36.48
Merc 230       22.8   4  95 36.48
Toyota Corona  21.5   4  97 34.40
Hornet 4 Drive 21.4   6 110 34.24
```

```
Volvo 142E     21.4   4 109 34.24
Mazda RX4      21.0   6 110 33.60
Mazda RX4 Wag  21.0   6 110 33.60
```

Much better with

```
pak::pak("hadley/genzplyr")
```

```
  Loading metadata database

  Loading metadata database ... done



  No downloads are needed

  1 pkg + 15 deps: kept 10 [29.8s]
```

```
library(genzplyr)
```

```
genzplyr loaded fr fr
Your data wrangling is about to be bussin no cap
```

```
mtcars |>
  yeet(mpg > 20) |>
  vibe_check(mpg, cyl, hp) |>
  glow_up(kpg = mpg * 1.6) |>
  slay(desc(mpg))
```

```
                mpg cyl  hp   kpg
Toyota Corolla 33.9   4  65 54.24
Fiat 128       32.4   4  66 51.84
Honda Civic    30.4   4  52 48.64
Lotus Europa   30.4   4 113 48.64
Fiat X1-9      27.3   4  66 43.68
Porsche 914-2  26.0   4  91 41.60
Merc 240D      24.4   4  62 39.04
Datsun 710     22.8   4  93 36.48
Merc 230       22.8   4  95 36.48
Toyota Corona  21.5   4  97 34.40
Hornet 4 Drive 21.4   6 110 34.24
Volvo 142E     21.4   4 109 34.24
Mazda RX4      21.0   6 110 33.60
Mazda RX4 Wag  21.0   6 110 33.60
```

```
# Complete analysis that's absolutely bussin
mtcars |>
  yeet(hp > 100) |>                    # Yeet the weak cars
  vibe_check(mpg, cyl, hp, wt) |>      # Vibe check our columns
  glow_up(                              # Glow up the data
```

```
    hp_per_ton = hp / (wt / 2),
    efficiency = mpg / (hp / 100)
  ) |>
  squad_up(cyl) |>                      # Squad up by cylinders
  no_cap(                                # Get the real stats
    avg_hp = mean(hp),
    avg_mpg = mean(mpg),
    avg_efficiency = mean(efficiency),
    squad_size = n()
  ) |>
  disband() |>                           # Disband the squads
  slay(desc(avg_efficiency)) |>          # Sort by slay factor
  send_it(10)
```

```
# A tibble: 3 x 5
    cyl avg_hp avg_mpg avg_efficiency squad_size
  <dbl>  <dbl>   <dbl>          <dbl>      <int>
1     4   111     25.9          23.3           2
2     6   122.    19.7          16.6           7
3     8   209.    15.1           7.67         14
```

A.6.3 Introduction to gpplot2

ggplot2 is based on ideas from the book "A grammar of graphics" by Leland Wilkinson and developed by Hadley Wickham, which also developed tidyverse.

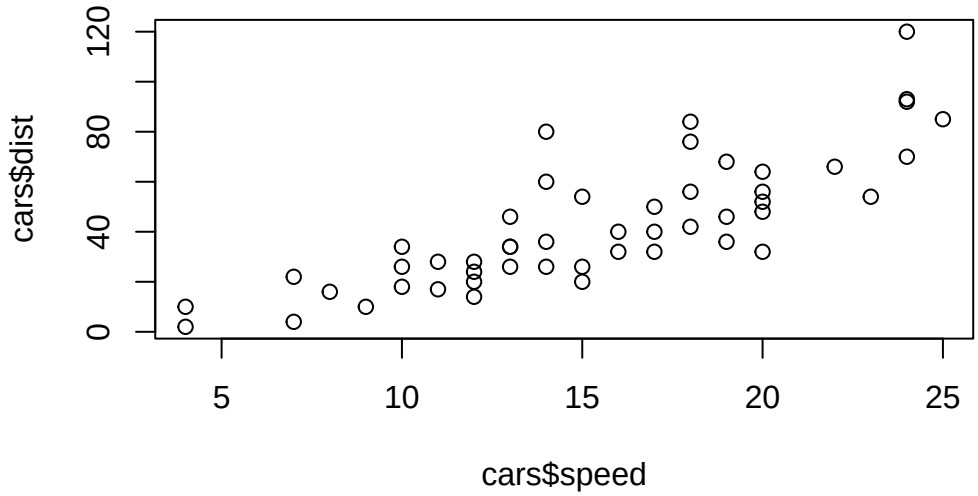
```
library(ggplot2)

cars
```

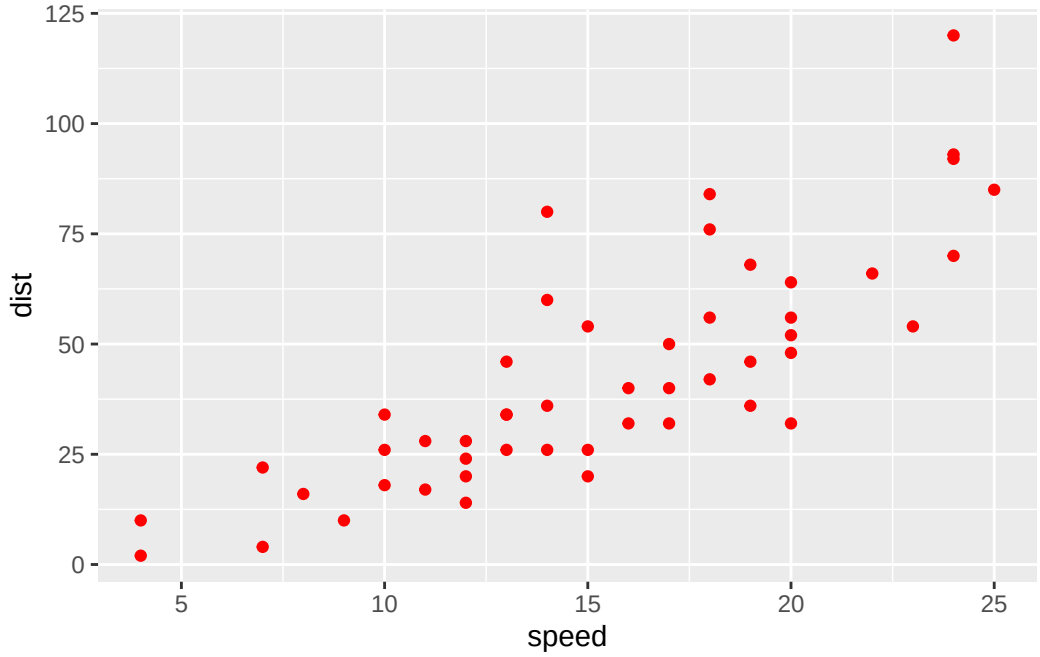
```
   speed dist
1      4    2
2      4   10
3      7    4
4      7   22
5      8   16
6      9   10
7     10   18
8     10   26
9     10   34
10    11   17
11    11   28
12    12   14
13    12   20
14    12   24
15    12   28
16    13   26
```

```
17    13   34
18    13   34
19    13   46
20    14   26
21    14   36
22    14   60
23    14   80
24    15   20
25    15   26
26    15   54
27    16   32
28    16   40
29    17   32
30    17   40
31    17   50
32    18   42
33    18   56
34    18   76
35    18   84
36    19   36
37    19   46
38    19   68
39    20   32
40    20   48
41    20   52
42    20   56
43    20   64
44    22   66
45    23   54
46    24   70
47    24   92
48    24   93
49    24  120
50    25   85
```

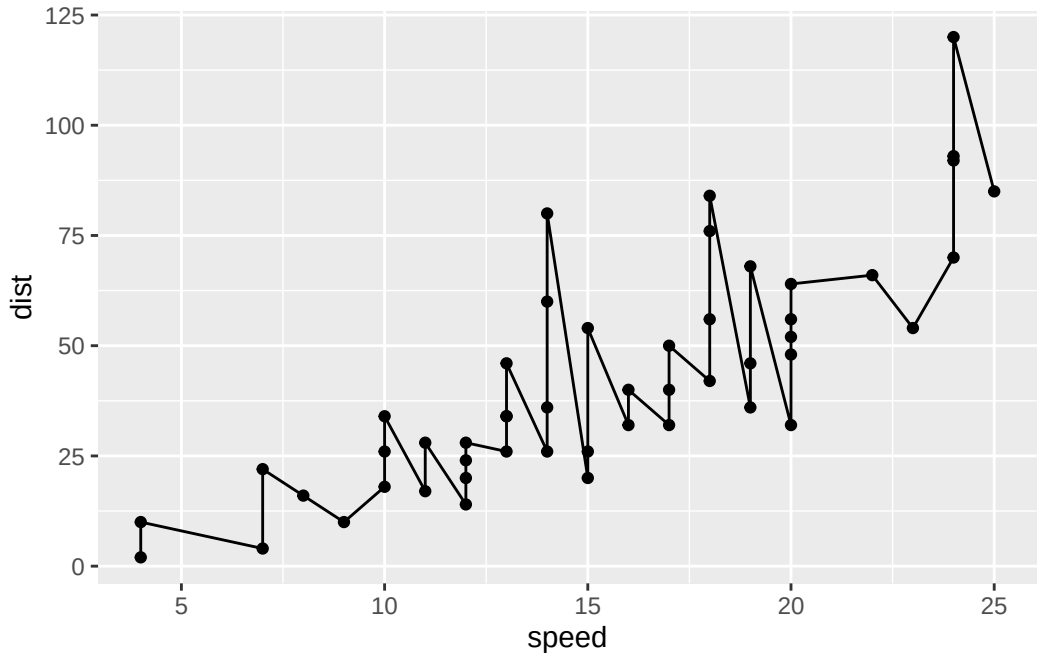
```
plot(cars$speed,cars$dist)
```




```
ggplot(data=cars, mapping=aes(x=speed,y=dist)) + geom_point(colour="red")
```

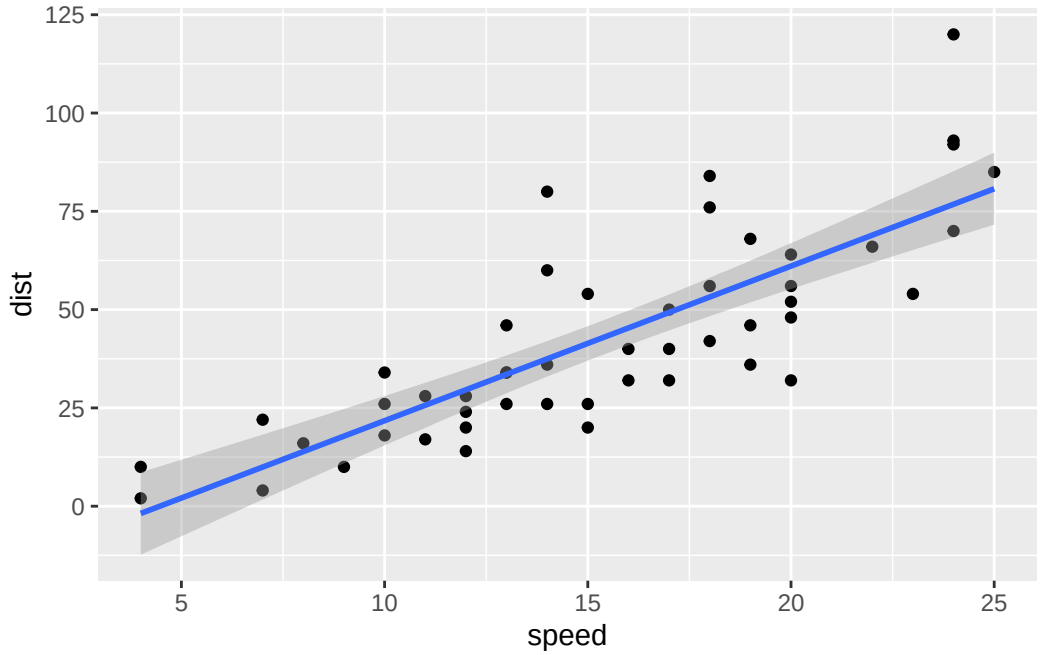



```
myplot <-  ggplot(cars, aes(speed,dist))+
  geom_point()+geom_line()
myplot
```



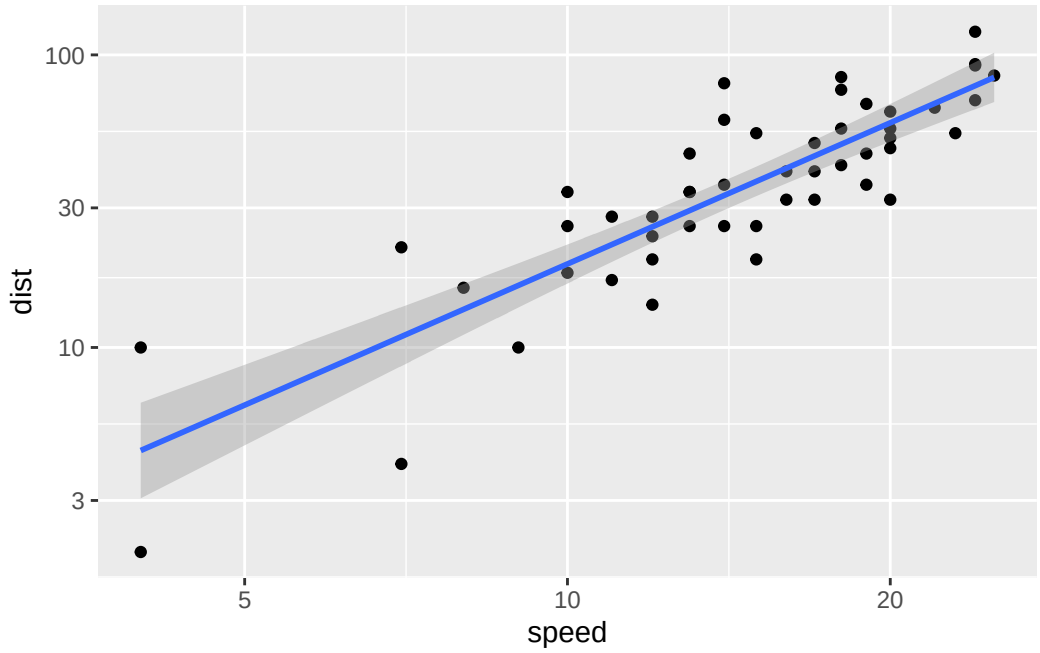

```
data(cars)
myplot <-  ggplot(cars, aes(speed,dist))+
  geom_point()+geom_smooth(method="lm")
myplot
```

```
`geom_smooth()` using formula = 'y ~ x'
```




```
data(cars)
myplot <-  ggplot(cars, aes(speed,dist))+
  geom_point()+geom_smooth(method="lm")+scale_x_log10()+scale_y_log10()
myplot
```

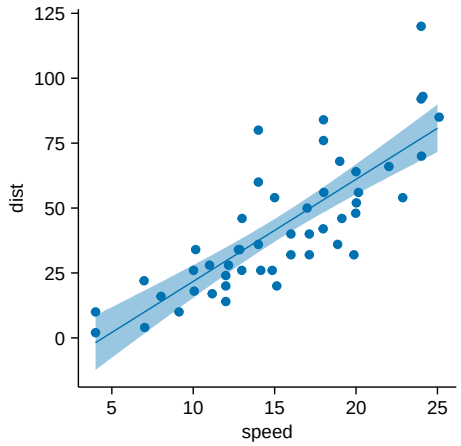
```
`geom_smooth()` using formula = 'y ~ x'
```



With tidyplot

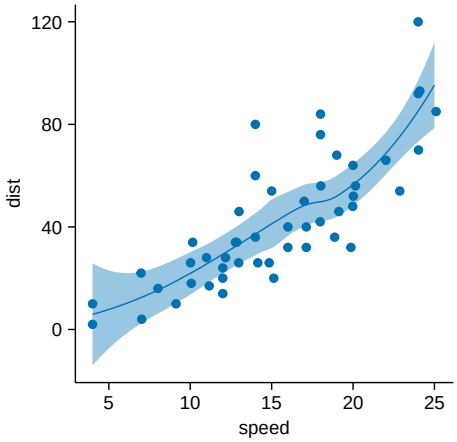
```
library(tidyplots)
cars |>  tidyplot(x=speed,y=dist) |>
  add_data_points_beeswarm() |>
  add_curve_fit(method="lm")
```

```
`geom_smooth()` using formula = 'y ~ x'
```




```
cars |>  tidyplot(x=speed,y=dist) |>
  add_data_points_beeswarm() |>
  add_curve_fit()
```

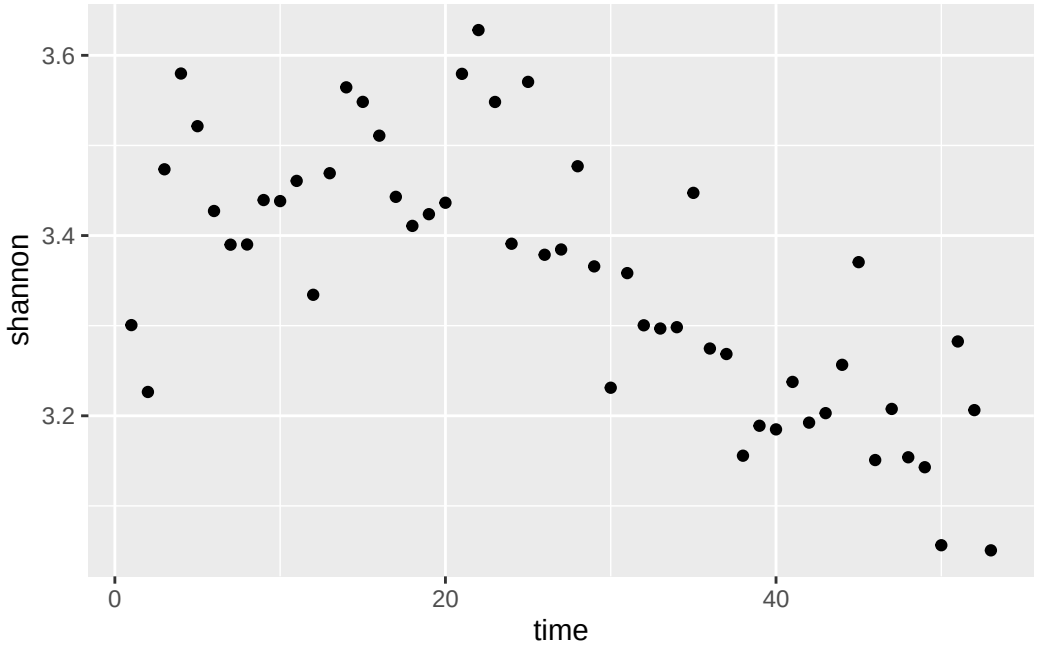
```
`geom_smooth()` using formula = 'y ~ x'
```



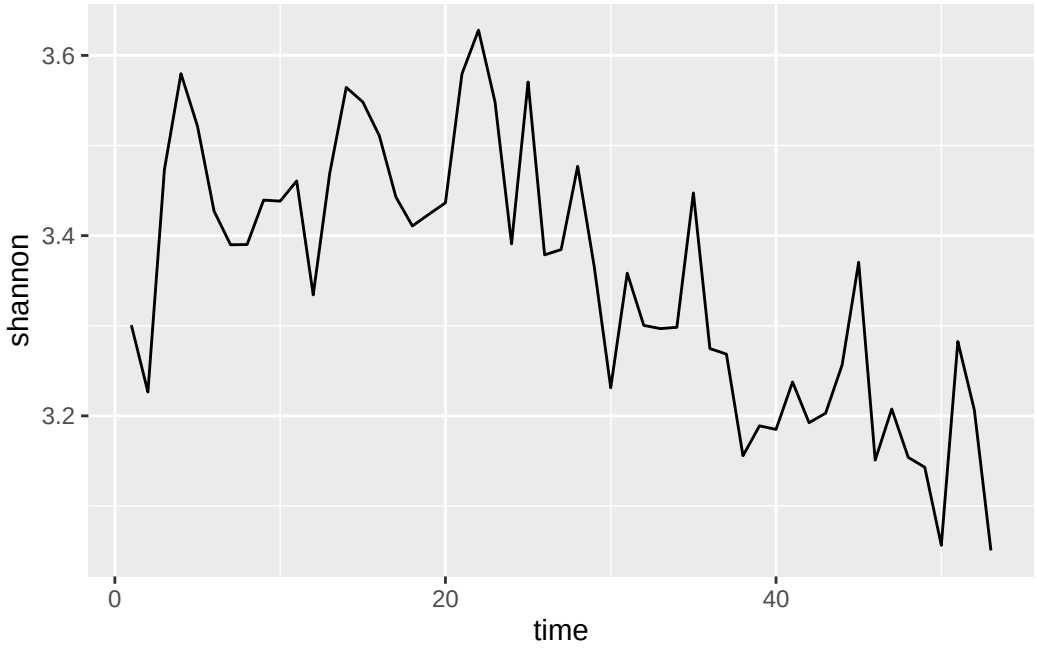
With the florida data

```
#ggplot
mat=cbind(t,out[10,])
colnames(mat)<-c("time","shannon")
mat<-as.data.frame(mat)

myplot <-  ggplot(mat, aes(time,shannon))+
  geom_point()
myplot
```



```
myplot <-  ggplot(mat, aes(time,shannon))+
  geom_line()
myplot
```



A.7 Working with biodiversity data: GBIF, EBV Portal

Authors: Corey Callaghan, Luise Quoss, Isabel Rosa.

First we load the library rgbif

```
library(rgbif)
library(tidyverse)
```

Now we will download observations of a species. Let's download observations of the common toad *Bufo bufo*.

```
matbufobufo<-occ_search(scientificName="Bufo bufo", limit=500, hasCoordinate = TRUE, hasGeospatialIs
```

```
Error in occ_search(scientificName = "Bufo bufo", limit = 500, hasCoordinate = TRUE, : could not fir
```

Let's examine the object *matbufobufo*

```
class(matbufobufo)
```

```
Error: object 'matbufobufo' not found
```

```
matbufobufo
```

```
Error: object 'matbufobufo' not found
```

It is a special object of class `gbif` which allows for the metadata and the actual data to all be included, as well as taxonomic hierarchy data, and media metadata. We won't worry too much about the details of this object now.

Let's download data about octupusses. They are in the order "Octopoda". First we need to find the GBIF search key for Octopoda.

```
a<-name_suggest(q="Octopoda",rank="Order")
```

```
Error in name_suggest(q = "Octopoda", rank = "Order"): could not find function "name_suggest"
```

```
key<-a$data$key
```

```
Error: object 'a' not found
```

We will only download 2000 observations to keep it simple for now. If you were doing this for real, you would download all data.

```
octopusses<-occ_search(orderKey=key,limit=2000, hasCoordinate = TRUE, hasGeospatialIssue = FALSE)
```

```
Error in occ_search(orderKey = key, limit = 2000, hasCoordinate = TRUE, : could not find function "o
```

Show the result

```
octmat<-octopusses$data
```

```
Error: object 'octopusses' not found
```

```
head(octmat)
```

```
Error: object 'octmat' not found
```

Count the number of observations per species using tidyverse and pipes

```
octmat %>%
  group_by(scientificName) %>%
  summarise(sample_size=n()) %>%
  arrange(desc(sample_size)) %>%
  mutate(sample_size_log=log(sample_size,2)) %>%
  ggplot(aes(x = sample_size_log)) + geom_histogram()
```

```
Error: object 'octmat' not found
```

Plot the records on an interactive map. First load the leaflet package.

```
library(leaflet)
leaflet(data=octmat) %>% addTiles() %>%
  addCircleMarkers(lat= ~decimalLatitude, lng = ~decimalLongitude,popup=~scientificName)
```

A.8 Working with raster and shapefiles

⚖️ Summary Table

Feature	`raster`	`terra`	`stars`
Core data type	`Raster*`	`SpatRaster`	`stars` array
Modernity	Legacy	Modern replacement	Alternative framework
Speed/memory	Slower	Fast, efficient	Moderate
Vector integration	`sp`	`sf`	`sf`
Dependencies	`sp`, `rgdal`, `rgeos`	none (GDAL/PROJ internal)	`sf`, `units`
Multidimensional (e.g. time)	Limited	Partial	Full
Recommended for new projects	❌	✅	✅ (if you need multi-dim or tidyverse integration)

Appendix B

Computational labs

B.1 Ecophysiology of an ectotherm

Solar radiation and convection are the two main pathways for animals such as small lizards. In this case, the total heat flux (f) into the animal is given by the **heat flux equation**:

$$f = q - h * (b - a)$$

- q = solar radiation (cal/h)
- $h * (b - a)$ = heat loss through convection
- b = body temperature of the animal (ºC)
- a = air temperature (ºC)
- h = convection heat transfer coefficient (cal/h/ ºC)

B.1.1 Plotting the climate space

a. Assuming that air temperature is 21ºC, and **h** = 60 cal/h, plot the lizard's equilibrium body temperature as a function of solar radiation. Assume that solar radiation varies between 0 and 1500 cal/h

b. Draw the climate space of the lizard by drawing the polygon that is limited by the minimum and maximum temperatures that the lizard can support, as a function of solar radiation. Consider that the upper lethal limit for the body temperature (***bmax***) is 36ºC and the lower lethal limit for the body temperature (***bmin***) is 24ºC.

c. Plot the following values of air temperature and solar radiation that are available throughout a day in two locations.

```
t <- c("00:00","03:00","06:00","09:00",    # Times of the day
   "12:00","15:00","18:00","21:00")
```

```
# Rock habitat
rock_q <- c(150,150,800,1100,1300,1200,800,400,150) # solar radiation
rock_a <- c(18,13,10,14,21,24,22,20,18)             # air temperature

# Bush habitat
bush_q <- c(150,150,450,600,650,650,350,200,150)    # solar radiation
bush_a <- c(18,13,10,14,21,24,22,20,18)             # air temperature
```

d. At which time of the day is the lizard on the rock, and at which time is it at the bush? Is there any time when the lizard cannot be at any of the locations?

B.1.2 Statistical modelling of body temperature

Section 1.4.2 presents a data frame containing data on individuals *A. cristatellus* living in urban and forested areas in three cities in Puerto Rico, giving us the perfect opportunity to assess the potential thermal effects of habitat upon this species. To do this:

a. Make a linear model for individuals of *A. cristatellus* living only in natural habitats (`context` = `natural`) and another for individuals living in urban habitats (`context` = `urban`).

Hint: to filter the data you can use the `subset` function in R

```
# example filtering data from anoles only from San Jose sites
df_sanjose <- subset(df, Site == "San Jose")
```

b. Is there a habitat where lizards are consistently warmer? If so, provide an ecological explanation (quantitatively) for this observation (assume an ambient temperature of 30 °C).

c. Is your model appropriate for explaining your data?

B.2 Optimal foraging theory

B.2.1 Predator with a strategy for maximizing energy *per time*.

Let a_i be the pontual abundance per unit of time of prey type i, e_i be the caloric content of prey type i, h_i be the time it takes to consume prey type i, e_w is the energy expended per unit of time by the predator while searching for prey, and e_h is the energy expended per unit of time by the predator to ingest prey.

Assume that there are only two types of prey and that:

- Strategy 1 – Chosing prey of type 1
- Strategy 2 – Chosing prey of type 2
- Strategy 3 – Chosing both preys

a) Show that if $e_1 > e_2$ and $h_1 < h_2$ then the second strategy is never a optimal strategy (assume $a_1 = a_2$).

b) Consider an active predator with the following physiological parameters:

$e_1 = 10$J; $e_2 = 100$J; $h_1 = 1$s; $h_2 = 60$s; $e_w = 1$ J/s; $e_h = 1$ J/S

Plot and comment the graph of the energy gain per unit of time for:

- the strategy of consuming both preys; and
- the strategy of consuming only the best prey.

Allow the abundances vary between 0.005 and 0.5 ind/s for the best prey. Consider the abundance of the worse prey to be 0.05 ind/s and 0.01 ind/s.

B.2.2 Predator with a strategy of sit-and-wait.

e_w is the energy expended per unit of time by the predator while waiting for its prey, e_p is the energy expended per unit of time by the predator while chasing the prey, v is the velocity of the predator in chase and a is the pontual abundance of preys per unit of time.

Consider the following predator and prey parameters:

$e_w = 0.1$ J/s; $e_p = 1$ J/S; $v = 0.5$ m/s; $a = 0.005$ ind/s/m2; and $e = 10$.

Produce a graph showing the energy gain per unit of time as a function of the size of the feeding territory.

B.3 Evolutionary games

B.3.1 1. The prisoners dilemma competition.

In this class we will have a competition of algorithms for playing the prisoners dilemma.

a) Write a function that takes as parameters two vectors that account with the history of the previous plays and returns one play (i.e., Cooperate - "C" or defect - "D"). This play should try to be the best response to the history of past moves. One example of such a function is:

```
strat<-function(own,opponent) #create a function with two arguments: my moves
                              # and my opponent's moves
{
    n<-length(opponent)   #check the number of moves from the opponent I know
    if(n==0) "D"          # If the opponent didn't play, I defect
    else
       if (own[n]=="D") opponent[n] # Else, if my last move was to defect,
                                    # I copy the last move of the opponent
       else "C"                     # Else, I cooperate
}
```

b) Use the function `pd_sim` (see below) to play your strategy with the strategy of the other groups (ask for each group their function(s)) and with itself, in an iterative game with X moves. Compare the results of the cumulative rewards as well as the sequence of moves. Report to the rest of groups the values of the rewards for each of the tournaments. What was the best strategy? What strategies are best responses to themselves?

```
pd_sim<-function(p1_strat,p2_strat,n)  # Be p1_strat and p2_strat any given strategy
{
  # you need to get a matrix (mat) of strategies

  w1<-0       #accumulated pay-off (fitness) of player 1
  w2<-0       #accumulated pay-offs of player 2
  h1<-NULL    #history of plays of player 1
  h2<-NULL    #history of plays of player 2

 for (t in 1:n)            # we will iterate the game for n times
    {
    a1<-p1_strat(h1,h2) # stores the play of player 1 based on the history of plays
    a2<-p2_strat(h2,h1) # stores the play of player 2 based on the history of plays
    p1<-mat[a1,a2]      # pay-off for player 1 of this play (based on the pay-off matrix)
    p2<-mat[a2,a1]      # pay-off for player 2 of this play (based on the pay-off matrix)
    w1<-w1+p1           # adds play's pay-off to the accumulated pay off of player 1
    w2<-w2+p2           # adds play's pay-off to the accumulated pay off of player 2
    h1[t]<-a1           # adds the play to the history of plays of player 1
    h2[t]<-a2           # adds the play to the history of plays of player 2
    }
  list(w1=w1,w2=w2,h1=h1,h2=h2)
    # returns the accumulate payoffs of player 1 and player 2,
```

```
    # and the history of plays of player 1 and 2
}
```

B.4 Dispersal and the random-walk

B.4.1 1. Random walk for 1 individual

a) Write your own code to simulate an individual random walk. Assume that the starting point is always $(x, y) = (0, 0)$ and the probability of an individual to choose any direction (i.e. left, right, up or down) is the same. It should be a function taking as argument the number of steps and returning a list of two vectors, one with the x positions over time and another with the y positions overtime.

b) Plot the random walks of several individuals on the same graph, using at least 5,000 steps for each individual.

B.4.2 2. Random walk for several (n) individuals

a) Write your own code to simulate random walks for n individuals and return the final position of each individual. The function should take the number of individuals and the number of time steps as arguments and return a list containing two vectors: one with the final x -coordinate of each individual and another with the final y -coordinate of each individual.

b) Create histograms showing the distributions of the final x - and y -coordinates of 10,000 individuals after 10, 100 and 1,000 time steps.

c) Create a function that takes a vector of values, `xlast`, a mean value, `meanx`, and a standard deviation, `stdx`, as arguments, and returns the log-likelihood of observing those values given those parameters.

d) Find the values `meanx` and `stdx` that maximize the likelihood of the observations. Are these values the same as `mean(xlast)` and `std(xlast)`? Explain why.

e) **Extra credit:** Create histograms showing the distribution of the distance from the origin

$$sqrt(x^2 + y^2)$$

for the final positions of 10,000 individuals after 10, 100, 1000 and 10000 time steps. How does the mean distance relate to the length of the random walk? Fit a Rayleigh distribution,

$$\left(\frac{r}{\sigma^2}\right) e^{-r^2/(2\sigma^2)}$$

to each histogram using non-linear fitting, and overlay the fitted distributions on the corresponding graphs.

B.5 Pandemic growth

B.5.1 1. The exponential dynamics of the pandemic.

In early 2020, the first wave of the global pandemic of COVID-19 hit several European countries. In this exercise, we will plot data for those countries and fit an exponential-growth model.

a) Load the time series of cumulative numbers of infected individuals during the COVID-19 pandemic in five countries. Each time series should begin when the first 15 infections have been detected.

Data source: https://github.com/owid/covid-19-data/tree/master/public/data 8 Nov 2020

b) Plot the data on a linear plot, using a different colour for each country. What do you observe?

c) Plot the data on a semi-logarithmic scale using the plotting option `log="y"`, again using a different colour for each country. What do you observe?

d) Perform a linear regression for each country using the logarithm of the n values, and estimate the growth rate R. Are the values of R the same for all countries? Do they vary over time?

e) Given that $R_0 = R^\tau$, and that the infectious period (τ) lasts 10 days, calculate R_0 for each countries?

f) Write a `for` loop to simulate geometric (exponential) growth over the time period `t = 1:100`. Store the population size at each time step `t+1` in variable `n`, based on the population size at time `t`, using `n[t+1]<-R*n[t]`. Remember to initialise the population size before the loop: `n<-1`. Plot a couple of runs of the loop using different values of R, such as $R = 1.01$ and $R = 1.1$, on linear scale.

g) The solution of the geometric growth equation is $n(t) = n_0 R^t$. Using the same values of R as above, create vectors using this formula, starting with `n0 <- 2`, and overlay them on the plot.

B.6 Population Viability Analysis

B.6.1 1. Expected time to extinction

Consider a population that grows exponentially but is subject to environmental stochasticity, with the logarithm of its growth rate following a normal distribution with mean $\bar{r} = \log(\bar{R})$ and variance v. Assume that the population has an on-off density dependence and cannot grow above the carrying capacity K. According to Foley (1994) the expected time to extinction in this population is:

$$T = \frac{1}{s \cdot r} \left[e^{s \log(K)} \left(1 - e^{-s \log(n_0)}\right) - s \log(n_0) \right],$$

where $s = 2r/v$.

a) Plot the mean extinction times, T, as a function of r, v, n_0, and K, commenting on each plot. Use the following ranges:

- r: 0.01 to 0.2 ;
- v: 0.05 to 1.0 ;
- n_0: 0 to 10 ;
- K: 10 to 500 .

Use $n_0 = 10$, $K = 100$, $v = 0.3$ and $r = 0.01$ as the baseline parameter values.

b) Write a function to simulate the population dynamics numerically. Begin with the `for` loop developed in Lab 5 and modify it to include a carrying capacity and a growth rate drawn annually from a normal distribution. The function should accept n_0, r, v, and K as parameter and return a vector containing the population size over time.

c) Simulate the population dynamics using the following parameter sets:

a. $n_0 = 3$; $r = 0.01$; $v = 0.2$; $K = 500$;
b. $n_0 = 100$; $r = 0.01$; $v = 0.2$; $K = 500$.

d) **Extended credit:** Compare the model predictions with the analytical approximation provided by Foley. Run a sufficiently large number of simulations to approach the predicted extinction times. For example, write a function that calls the function developed in part (b) 100 times and returns the median time to extinction across the simulations.

B.7 Reference

Foley, P. (1994) Predicting Extinction Times from Environmental Stochasticity and Carrying-Capacity. *Conservation Biology* **8**: 124–137.

B.8 The camera trapper

A researcher place camera traps at a grid of sites during 19 days to observe roe-deer.

B.8.1 1. Load the data on the file roe_deer_2016_r.xls into R and briefly describe the structure of the dataset.

B.8.2 2. Case 1: Ignoring detection probability.

a) Construct a vector that for each site takes value 1 when roe deer is present and 0 when is absent.

b) Use maximum likelihood to estimate the occupancy probability, Ψ. First, create a function that takes Ψ and a vector of presence-absence data as arguments and returns the likelihood. Then, plot this function over a range of Ψ values and determine the value that maximizes the likelihood.

c) Based on previous research, assume that the occupancy is somewhat between 0.2 and 0.5. Using a Bayesian approach, calculate the posteriori probability distribution for the occupancy $P(\Psi)$.

```
Set function for Bayesian approach:
occupancybayesian <- function(v,psi,priorpsi)
{
            integrand <- function(x)
                occupancylikelihood(v,x)*priorpsi(x)
occupancylikelihood(v,psi)*priorpsi(psi)/
integrate(integrand, lower = 0.01, upper = 0.99)$value
}

Set function for uninformed prior:
uninformedprior <- function(psi)
{
  dunif(psi,0.2,0.5)
}
```

B.8.3 3. Use maximum-likelihood estimate both the occupancy probability, Ψ, and the detection probability, p.

B.9 Managing a fishery

B.9.1 1. Consider that you are managing a fishery with the following dynamics:

$$\frac{dn}{dt} = 0.2n\left(1 - \frac{n}{10000}\right)$$

a) Plot the production function of the fishery, indicating the stock size at which the maximum sustainable yield occurs and the corresponding production level.

b) Suppose you establish an annual quota equivalent to 50% of the maximum sustainable yield. What are the two stock-size levels that sustainably allow this exploitation level?

c) Consider an annual variance of 10% in productivity. Explore what happens when you manage this fishery for 100 years at each of the two stock sizes determined in the previous question.

d) Explain why, in a system with constant quotas, MSY is not stable and should not be the management goal.

e) Discuss how these results relate to the paper by Worm et al. (2009) on the need to rebuild fisheries.



Figure B.1: Atlantic cod (*Gadus morhua*)

B.10 Road kill

B.10.1 The *Red fox*

…So, while they may be most active at night. Red foxes are especially active during the daytime in ***spring and summer (April-Sept)*** *as they are foraging for food to feed their young. …*



B.10.2 1. Install the following R packages:

```
library(readxl)
library(dplyr)
library(sf)
library(ggplot2)
library(fuzzySim)
library(grid)
```

B.10.3 2. Identify road segments with hotspots

a) Load the roadkill dataset and explain its structure.

b) Show that differences in seasonal activity patterns affect the number of red fox–car collisions.

c) Using the False Discovery Rate (fdr) method, how many hotspots were detected for the active season?

d) Based on the FDR method, plot the true and "false" hotspots. Explain the general geographic locations of the hotspots and how they vary between the two seasons.

e) Consider the implications for conservation management.

B.11 The unified neutral theory of biodiversity and biogeography

B.11.1 1. Understanding neutral dynamics

a) The four histograms labeled "a" through "d" represent the species abundance distributions of four local communities, each containing 1,600 individuals. All communities follow the dynamics of the neutral theory under the point mutation speciation model. The corresponding metacommunity is identical for all cases; therefore, the only factor differing among these four local communities is the probability of immigration.

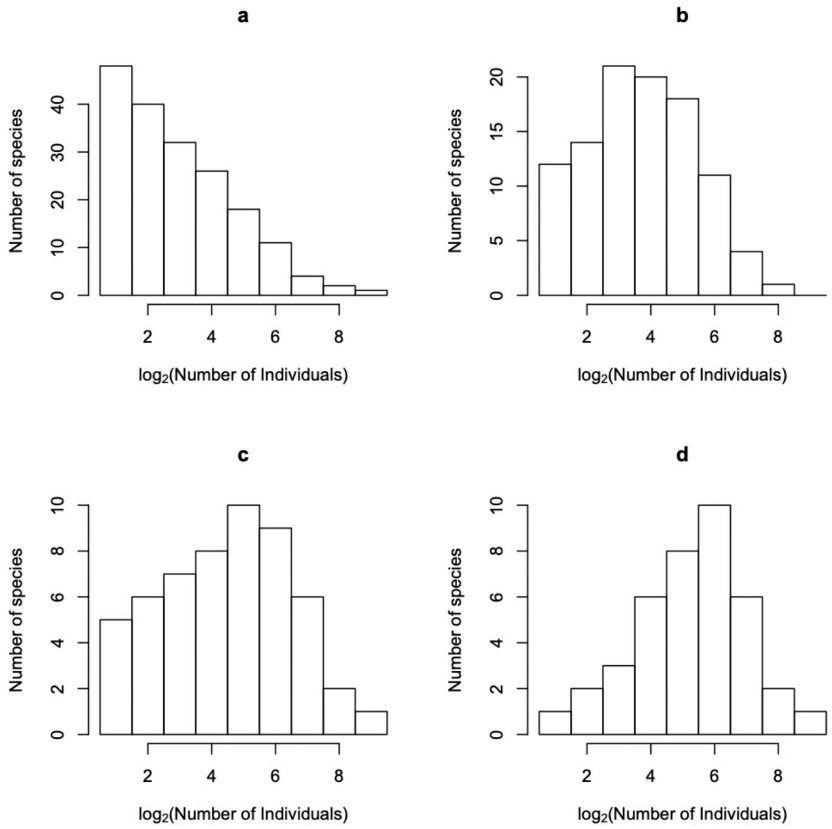


Figure B.2: Species-abundance distributions

Identify the communities in order of increasing immigration probability and explain your reasoning.

---

b) Draw the logseries (metapopulation SAD) distribution for several values of θ (e.g., 0.1, 1, 10); assume $x = 0.99944$ and $n = 1 : 10000$. Draw the curves in linear-linear, log-linear and log-log

scales and discuss the influence of θ for the shape of the distribution. Recall that the logseries is given by

$$S = \theta \frac{x^n}{n}$$

---

c) Using the R function `vallade.houchmandzadeh.eq` plot the S.A.D. for local communities with $J = 16,000$ individuals and different values of θ (e.g., 5, 50) and m (e.g. 0.001, 0.01, 0.1) values. Discuss how different values of θ and m affect the distribution, and explain why.

Warnings:

- The function is slow;
- In some cases (typically for small values of) you may get an error message!

```
vallade.houchmandzadeh.eq <- function(J=16000, theta=50, m=0.01){

# vallade.houchmandzadeh.eq integral of Vallade and Houchmandzadeh's equation
# equivalent to equation 7 of Volkov et al.
#
# Attention: this function hasn't been fully tested for m=1

### Auxiliary functions:

#######################################################################

# function der.eq.7
# derivative of equation (15) of Vallade and Houchmandzadeh
# Attention we are using the same variables as Volkov, therefore we need to
# introduce the terms corresponding to the chain of variables:
#   x = mu * w

der.eq.7 <- function(x)
{
   #mu <- m * (J-1) / (1-m)

# to avoid overflow we work with logs and only convert at the end

   der.equation.7 <- ( log(theta) + cte
                  + (theta-1)*log(1-x) - log(x)
                  + log.fract.gamma(n+x*mu, x*mu)
                  + log.fract.gamma(J-n+mu*(1-x), mu*(1-x)) )

   exp(der.equation.7)

}
```

```
#########################################################################

# function log.fract.gamma calculates the ln of the fraction with gamma
#           functions in equation (7)

log.fract.gamma <- function(a, b)
{

  inside1 <- function(aa) {
    if(aa <= 1) log(gamma(aa))
    else log.stirling(aa)
  }

  log.num <- sapply(a,inside1)

  inside2 <- function(bb) {
    if(bb <= 1) log.den <- log(gamma(bb))
    else log.den <- log.stirling(bb)
  }
  log.den <- sapply(b,inside2)

  fraction <- log.num - log.den

  return(fraction)
}

#########################################################################

# function log.stirling
# calculates the log of the stirling approximation of the gamma function

log.stirling <- function(x)
{
   pi <- 3.1415926
   pi.t <- 0.5 * log(2*pi)

   extra <- log(1 + 1/12/x + 1/288/x/x -139/51840/x^3 + 571/2488320/x^4)
   log.st <- -x + (x - 0.5)*log(x) + pi.t + extra

   return(log.st)
}

#########################################################################

bin.nospp <- function(data){
```

```
# this function bins data assuming that the data is in the format
# line 1 - number of species with one individual
# line 2 - number of species with two individuals
# line 3 - etc.

        #quartz(width=9,height=7)
        #layout(matrix(c(1,2,3),3,1))

  NN <- 40  # no of bins
    no.p <- length(data)

  n    <- 1:NN
    bin <- rep(0,NN)
    pow2 <- 2^n

    ini <-  1
  nfin <- trunc(log(no.p)/log(2) + 1)
  for (i in 1:nfin){
        fin <- pow2[i]-1
                if(fin > no.p) fin <- no.p
        bin[i] <- sum(data[ini:fin])
                if(fin > no.p) break
        ini <- fin + 1
    }
    y <- bin[bin>0]
    x <- 1:length(y)

    plot(x, y, xlab=substitute(paste(log[2],"(Number of individuals)",sep="") ),
                      ylab="Number of species",
                   main="Hubbell's binning method",
                   type="h", lwd=30, lend=1)

  legend("topright",c(paste("J=",J),as.expression(substitute(theta==tt,list(tt=theta))),paste("m=",m

  lines(x,y,col="red")
}

####### End of auxiliary functions #######

  res.int <- numeric(J)
  J <<- J
  theta <<- theta
  m <<- m

    if(m==1){
```

```
        for(i in 1:J){
      f1 <- log.fract.gamma(J+1,J+1-i)
      f2 <- log.fract.gamma(J+theta-i,J+theta)
      res.int[i] <- theta/i*exp(f1+f2)
    }
  }
    else{
    mu <<- m * (J-1) / (1-m)
    ct1 <- log.fract.gamma(mu, J+mu)

        for(i in 1:J){

      n <<- i
      cte <<- (ct1 + log.fract.gamma(J+1, J-n+1)
                                    - log.stirling(n+1) )
      res <- integrate(der.eq.7, 0, 1)
      res.int[i] <- res[[1]]
    }
    }

  bin.nospp(res.int)
}
```

---

d) **Extra credit** The exercise is based on https://hankstevens.github.io/Primer-of-Ecology/diversity.html#neutral-theory-of-biodiversity-and-biogeography

1) Install and upload the package `untb`:

```
install.packages("untb")
library(untb)
```

Read the instructions on the package and briefly describe the information inside it.

---

2)

If you type

```
a <- untb(start=rep(1:20, 25), prob=0, gens=2500, D=9, keep=TRUE)
```

you will get a matrix, describe the data found in the matrix.

Plot the results to see how the identities change through time at time steps 1, 100, 2500. A single point represents an individual and the different colours represent the different identities, and create the three graphs at the three time points, with the appropriate data, colours, and titles.

---

3) Describe what you obtained in the three previous plots.

---

4) Plot the abundance distribution using the following instructions.

```
layout(matrix(1:3,ncol=3))

par(mar=c(5,4,1,1))
for(i in times){
plot(as.count(a[i,]),
  ylim=c(1,max(as.count(a[2000,]))), xlim=c(0,20))
}

par(mar=c(6,4,1,3))
out <- lapply(times, function(i)  preston(a[i,]) )
bins <- matrix(0, nrow=3, ncol=length(out[[3]]) )
for(i in 1:3) bins[i,1:length(out[[i]])] <- out[[i]]
#bins

colnames(bins) <- names(preston(a[times[3],]))
for(i in 1:3){
  par(las=2)
  barplot(bins[i,], ylim=c(0,20), xlim=c(0,8), ylab="No. of Species",
          xlab="Abundance Category" )
}
```

Describe the species rank abundance distributions and the relationship between the number of species and abundance category.

B.11.2 References:

Hubbell, S.P. (2001). The Unified Neutral Theory of Biodiversity and Biogeography. Princeton University Press. ISBN 9780691021287.

Rosindell et al (2011). The Unified Neutral Theory of Biodiversity and Biogeography at Age Ten. *Trends in Neutral Ecology & Evolution*, **26(7)**. https://doi.org/10.1016/j.tree.2011.03.024

Stevens, M. H. 2009. A Primer of Ecology with R. First edition. Springer.

B.12 Monitoring biodiversity

B.12.1 1. Exploring the North American Breeding Bird Survey (BBS) dataset

a) Load the file **Florida.csv** into R and briefly describe the structure of the dataset.

b) Map the monitored transects/routes in Florida. Use the **maps** library to plot Florida's counties as a base map.

c) For **transects 4** and **109** in 2018, calculate the species richness, Shannon index, $H^{'}$, and evenness J for both transects, where

$$H^{'} = -\sum p_i \ln p_i$$

and p_i is the proportion of individuals belonging to species i. Compare the results for the two transects.

d) Plot the species-abundance distributions and abundance ranks for both transects. What do you observe?

e) Choose one transect and plot its species richness, Shannon index, and geometric mean abundance over time.

f) **Extra credit 1:** Produce a map showing the trends in one of these metrics across Florida, for instance, by coloring different points according to the trend.

g) **Extra credit 2:** Estimate the number of species in Florida by combining the different transects and using one of the available estimators. Use bootstrap and/or jacknife to calculate confidence intervals.